

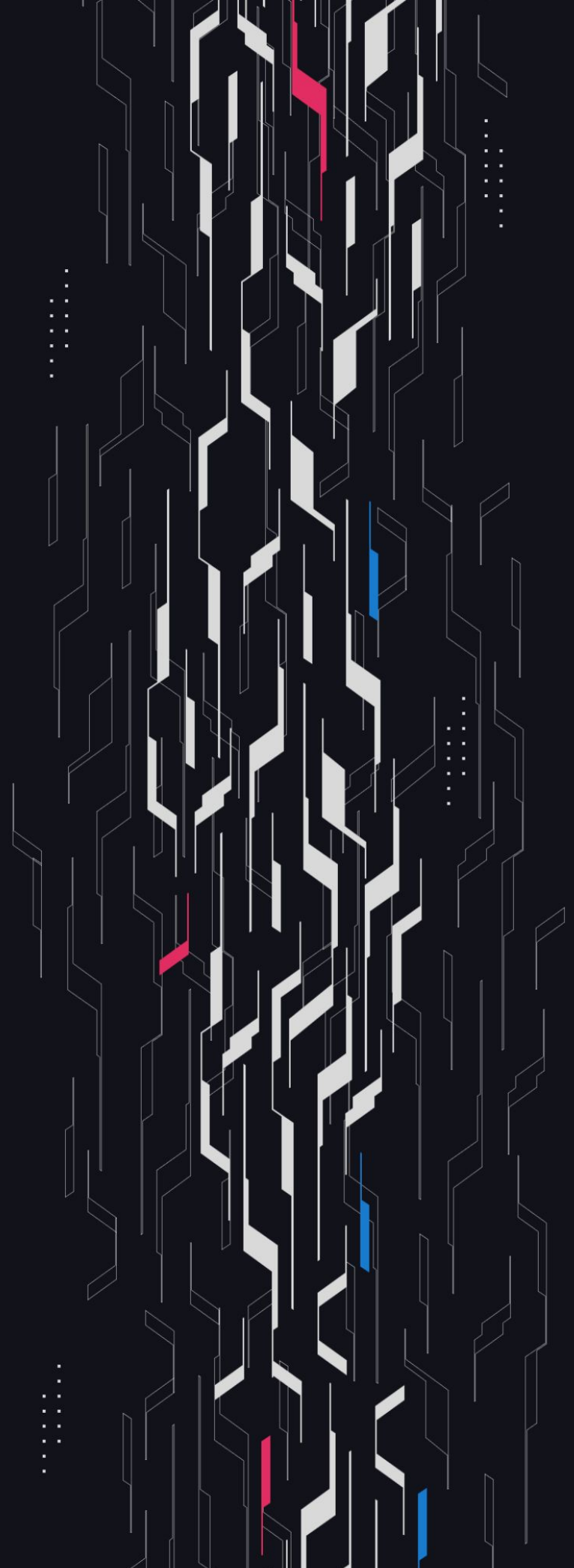
GA GUARDIAN

GMX

V2

Security Assessment

July 8th, 2026



Summary

Audit Firm Guardian

Prepared By CuriousApple, Nicholas Chew, Cosine,

Client Firm GMX

Final Report Date July 8, 2026

Audit Summary

GMX engaged Guardian to review the security of GMX V2. From January 19, 2026 through June 12, 2026, Guardian reviewed the in-scope contracts across the main review and four follow-up rounds, including remediation changes through the final commit.

Confidence Ranking

Given the lack of High and Critical issues detected during the main review, Guardian assigns a Confidence Ranking of 4 to the protocol. Guardian advises the protocol to consider periodic review with future changes. For detailed understanding of the Guardian Confidence Ranking, please see the rubric on the following page.

✓ Verify the authenticity of this report on Guardian's GitHub: <https://github.com/guardianaudits>

Guardian Confidence Ranking

Confidence Ranking	Definition and Recommendation	Risk Profile
5: Very High Confidence	Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. Recommendation: Code is highly secure at time of audit. Low risk of latent critical issues.	0 High/Critical findings and few Low/Medium severity findings.
4: High Confidence	Code is clean, well-structured, and adheres to best practices. Only 1 Significant issue was uncovered per week. Design patterns are sound, and test coverage is strong. Recommendation: Suitable for deployment after remediations; consider periodic review with changes.	0-1 High/Critical findings per engagement week and little to no Medium severity issues. Varied Low severity findings.
3: Moderate Confidence	Medium-severity and occasional High-severity issues found. Code is functional, but there are concerning areas (e.g., weak modularity, risky patterns). No critical design flaws, though some patterns could lead to issues in edge cases. Recommendation: Address issues thoroughly and consider a targeted follow-up audit depending on code changes.	1-2 High/Critical findings per engagement week.
2: Low Confidence	Code shows frequent emergence of Critical/High vulnerabilities. Audit revealed recurring anti-patterns, weak test coverage, or unclear logic. These characteristics suggest a high likelihood of latent issues. Recommendation: Post-audit development and a second audit cycle are strongly advised.	2-4 High/Critical findings per engagement week. Or additional High/Critical findings uncovered in remediation review which have not been resolved and confirmed by Guardian.
1: Very Low Confidence	Code has systemic issues. Multiple High/Critical findings (≥ 5/week), poor security posture, and design flaws that introduce compounding risks. Safety cannot be assured. Recommendation: Halt deployment and seek a comprehensive re-audit after substantial refactoring.	≥ 5 High/Critical findings and overall systemic flaws.

Table of Contents

Project Information

Project Overview 5

Audit Scope & Methodology 6

Smart Contract Risk Assessment

Findings & Resolutions 11

Addendum

Disclaimer 163

About Guardian 164

Project Overview

Project Summary

Project Name	GMX
Language	Solidity
Codebases	https://github.com/gmx-io/gmx-synthetics https://github.com/gmx-io/gmx-synthetics-private https://github.com/gmx-io/gmx-functions
GMX Synthetics Commits	Initial: 256a058fa996d8da5bea9eb617557e323d7379da Final: 1b08b014f9435a34d6e669ff206051954f8a9461
GMX Functions Commit	007d7b8e9391fb53ac3e27e501d5e7d08faa210d

Audit Summary

Delivery Date	July 8, 2026
Audit Methodology	Static Analysis, Manual Review

Vulnerability Summary

Vulnerability Level	Total	Pending	Declined	Acknowledged	Partially Resolved	Resolved
● Critical	2	0	0	0	0	2
● High	7	0	0	1	0	6
● Medium	31	0	0	4	0	27
● Low	21	0	0	9	0	12
● Info	23	0	0	6	0	17

Audit Scope & Methodology

contracts/ **order**

AutoCancelUtils.sol
BaseOrderUtils.sol
DecreaseOrderExecutor.sol
DecreaseOrderUtils.sol
ExecuteOrderUtils.sol
IBaseOrderUtils.sol
IncreaseOrderExecutor.sol
IncreaseOrderUtils.sol
Order.sol
OrderEventUtils.sol
OrderStoreUtils.sol
OrderUtils.sol
OrderVault.sol
SwapOrderExecutor.sol
SwapOrderUtils.sol

contracts/ **exchange**

AdlHandler.sol
BaseHandler.sol
BaseOrderHandler.sol
DepositHandler.sol
GlvDepositHandler.sol
GlvShiftHandler.sol
GlvWithdrawalHandler.sol
IDepositHandler.sol
IGlvDepositHandler.sol
IGlvWithdrawalHandler.sol
IJitOrderHandler.sol
IOrderExecutor.sol
IOrderHandler.sol
IShiftHandler.sol
IWithdrawalHandler.sol
JitOrderHandler.sol
LiquidationHandler.sol
OrderHandler.sol
ShiftHandler.sol
WithdrawalHandler.sol

contracts/ **deposit**

Deposit.sol
DepositEventUtils.sol
DepositStoreUtils.sol
DepositUtils.sol
DepositVault.sol
ExecuteDepositUtils.sol
IDepositUtils.sol
IExecuteDepositUtils.sol

contracts/ **withdrawal**

ExecuteWithdrawalUtils.sol
IExecuteWithdrawalUtils.sol
IWithdrawalUtils.sol
Withdrawal.sol
WithdrawalEventUtils.sol
WithdrawalStoreUtils.sol
WithdrawalUtils.sol
WithdrawalVault.sol

contracts/ **shift**

IShiftUtils.sol
Shift.sol
ShiftEventUtils.sol
ShiftStoreUtils.sol
ShiftUtils.sol
ShiftVault.sol

contracts/ **swap**

ISwapHandler.sol
ISwapUtils.sol
SwapHandler.sol
SwapUtils.sol

contracts/ **claim**

ClaimEventUtils.sol
ClaimHandler.sol
ClaimUtils.sol
ClaimVault.sol

contracts/ **market**

Market.sol
MarketEventUtils.sol
MarketFactory.sol
MarketPoolValueInfo.sol
MarketStoreUtils.sol
MarketToken.sol
MarketUtils.sol
PositionImpactPoolUtils.sol

contracts/ **position**

DecreasePositionCollateralUtils.sol
DecreasePositionSwapUtils.sol
DecreasePositionUtils.sol
IncreasePositionUtils.sol
Position.sol
PositionEventUtils.sol
PositionStoreUtils.sol
PositionUtils.sol

contracts/ **pricing**

ISwapPricingUtils.sol
PositionPricingUtils.sol
PricingUtils.sol
SwapPricingUtils.sol

contracts/ **fee**

FeeBatch.sol
FeeBatchStoreUtils.sol
FeeDistributor.sol
FeeDistributorUtils.sol
FeeDistributorVault.sol
FeeHandler.sol
FeeSwapUtils.sol
FeeUtils.sol

contracts/ **gas**

GasUtils.sol

Audit Scope & Methodology

contracts/ **liquidation**

LiquidationUtils.sol

contracts/ **adl**

AdlUtils.sol

contracts/ **glv**

Glv.sol
GlvEventUtils.sol
GlvFactory.sol
GlvStoreUtils.sol
GlvToken.sol
GlvUtils.sol
GlvVault.sol

contracts/ **glv/glvDeposit**

ExecuteGlvDepositUtils.sol
GlvDeposit.sol
GlvDepositCalc.sol
GlvDepositEventUtils.sol
GlvDepositStoreUtils.sol
GlvDepositUtils.sol
IGlvDepositUtils.sol

contracts/ **glv/glvWithdrawal**

GlvWithdrawal.sol
GlvWithdrawalEventUtils.sol
GlvWithdrawalStoreUtils.sol
GlvWithdrawalUtils.sol
IGlvWithdrawalUtils.sol

contracts/ **glv/glvShift**

GlvShift.sol
GlvShiftEventUtils.sol
GlvShiftStoreUtils.sol
GlvShiftUtils.sol

contracts/ **data**

DataStore.sol
Keys.sol
Keys2.sol

contracts/ **config**

AutoCancelSyncer.sol
Config.sol
ConfigSyncer.sol
ConfigTimelockController.sol
ConfigUtils.sol
IRiskOracle.sol
TimelockConfig.sol

contracts/ **oracle**

ChainlinkDataStreamProvider.sol
ChainlinkPriceFeedProvider.sol
ChainlinkPriceFeedUtils.sol
EdgeDataStreamProvider.sol
EdgeDataStreamVerifier.sol
GmOracleProvider.sol
GmOracleUtils.sol
IChainlinkDataStreamVerifier.sol
IOracle.sol
IOracleProvider.sol
IPriceFeed.sol
Oracle.sol
OracleModule.sol
OracleStore.sol
OracleUtils.sol

contracts/ **multichain**

BridgeOutFromControllerUtils.sol
IMultichainGlvRouter.sol
IMultichainGmRouter.sol
IMultichainOrderRouter.sol
IMultichainProvider.sol
IMultichainTransferRouter.sol
IOriginator.sol
LayerZeroProvider.sol
MultichainClaimsRouter.sol
MultichainEventUtils.sol
MultichainGlvRouter.sol
MultichainGmRouter.sol
MultichainOrderRouter.sol
MultichainReader.sol
MultichainReaderUtils.sol
MultichainRouter.sol
MultichainSubaccountRouter.sol
MultichainTransferRouter.sol
MultichainUtils.sol
MultichainVault.sol

contracts/ **router**

BaseRouter.sol
ExchangeRouter.sol
GlvRouter.sol
IExchangeRouter.sol
Router.sol
SubaccountRouter.sol
SimulationRouter.sol

contracts/ **router/relay**

BaseGelatoRelayRouter.sol
GelatoRelayRouter.sol
IRelayUtils.sol
RelayUtils.sol
SubaccountGelatoRelayRouter.sol
SubaccountRouterUtils.sol

Audit Scope & Methodology

contracts/ reader

GlvReader.sol
KeeperReader.sol
Reader.sol
ReaderDepositUtils.sol
ReaderPositionUtils.sol
ReaderPricingUtils.sol
ReaderUtils.sol
ReaderWithdrawalUtils.sol

contracts/ utils

AccountUtils.sol
Array.sol
BasicMulticall.sol
Bits.sol
Calc.sol
Cast.sol
EnumerableValues.sol
GlobalReentrancyGuard.sol
PayableMulticall.sol
Precision.sol
Printer.sol
StringUtils.sol
Uint256Mask.sol

contracts/ callback

CallbackUtils.sol
IDepositCallbackReceiver.sol
IGasFeeCallbackReceiver.sol
IGlvDepositCallbackReceiver.sol
IGlvWithdrawalCallbackReceiver.sol
IOrderCallbackReceiver.sol
IShiftCallbackReceiver.sol
IWithdrawalCallbackReceiver.sol

contracts/ referral

IGov.sol
IReferralStorage.sol
ITimelock.sol
ReferralEventUtils.sol
ReferralTier.sol
ReferralUtils.sol

contracts/ chain

ArbGasInfo.sol
ArbSys.sol
Chain.sol
ChainReader.sol

contracts/ role

Role.sol
RoleModule.sol
RoleStore.sol

contracts/ gov

GovTimelockController.sol
GovToken.sol
ProtocolGovernor.sol

contracts/ token

IWNT.sol
TokenUtils.sol

contracts/ bank

Bank.sol
StrictBank.sol

contracts/ event

EventEmitter.sol
EventUtils.sol

contracts/ error

ErrorUtils.sol
Errors.sol

contracts/ external

ExternalHandler.sol
IExternalHandler.sol

contracts/ price

Price.sol

contracts/ nonce

NonceUtils.sol

contracts/ feature

FeatureUtils.sol

contracts/ safe

SafeUtils.sol

contracts/ subaccount

SubaccountUtils.sol

Audit Scope & Methodology

Vulnerability Classifications

Severity	Impact: <i>High</i>	Impact: <i>Medium</i>	Impact: <i>Low</i>
Likelihood: <i>High</i>	● Critical	● High	● Medium
Likelihood: <i>Medium</i>	● High	● Medium	● Low
Likelihood: <i>Low</i>	● Medium	● Low	● Low

Impact

- High**

Significant loss of assets in the protocol, significant harm to a group of users, or a core functionality of the protocol is disrupted.
- Medium**

A small amount of funds can be lost or ancillary functionality of the protocol is affected. The user or protocol may experience reduced or delayed receipt of intended funds.
- Low**

Can lead to any unexpected behavior with some of the protocol's functionalities that is notable but does not meet the criteria for a higher severity.

Likelihood

- High**

The attack is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount gained or the disruption to the protocol.
- Medium**

An attack vector that is only possible in uncommon cases or requires a large amount of capital to exercise relative to the amount gained or the disruption to the protocol.
- Low**

Unlikely to ever occur in production.

Audit Scope & Methodology

Methodology

Guardian is the ultimate standard for Smart Contract security. An engagement with Guardian entails the following:

- Two competing teams of Guardian security researchers performing an independent review.
- A dedicated fuzzing engineer to construct a comprehensive stateful fuzzing suite for the project.
- An engagement lead security researcher coordinating the 2 teams, performing their own analysis, relaying findings to the client, and orchestrating the testing/verification efforts.

The auditing process pays special attention to the following considerations:

- Testing the smart contracts against both common and uncommon attack vectors.
- Assessing the codebase to ensure compliance with current best practices and industry standards.
- Ensuring contract logic meets the specifications and intentions of the client.
- Cross-referencing contract structure and implementation against similar smart contracts produced by industry leaders.
- Thorough line-by-line manual review of the entire codebase by industry experts.
Comprehensive written tests as a part of a code coverage testing suite.
- Contract fuzzing for increased attack resilience.

Main Review Findings & Resolutions

ID	Title	Category	Severity	Status
C-01	Reserve check dangerously blocks decrease flows	Logical Error	● Critical	Resolved
C-02	factory.call() enables unauthorized transfers	Signatures	● Critical	Resolved
H-06	depositReferralRewards Call Would Always Revert	Rounding	● High	Resolved
H-07	Wrong Operation Order Cause Reward Loss	Logical Error	● High	Resolved
H-10	Non-Determinism Issues In Chainlink CRE Workflow	Logical Error	● High	Acknowledged
H-12	Reserve checks can block liquidations and ADLs	Logical Error	● High	Resolved
H-13	StaticOracleProvider returns min/max inverted	DoS	● High	Resolved
M-01	Incompatible Gas Assumptions On MegaEVM	Configuration	● Medium	Resolved
M-07	validateGlvShiftMaxLoss Uses Post-Shift Input	Logical Error	● Medium	Resolved
M-10	Risk Free Trades For Malicious Keeper	Validation	● Medium	Resolved
M-11	Event Lookup Across Batch Can Cause Failures	DoS	● Medium	Resolved
M-17	V2 Keeper Costs Can Underflow Calculations	DoS	● Medium	Resolved
M-31	Atomic Oracle Providers Enable Price Picking	Logical Error	● Medium	Resolved

Main Review Findings & Resolutions (continued)

ID	Title	Category	Severity	Status
M-32	GLV market token deposits bypass expiry	Logical Error	● Medium	Resolved
M-33	Edge data stream decimals are not validated	Validation	● Medium	Resolved
M-34	Missing Chainlink ref price feeds for tokens	Configuration	● Medium	Resolved
M-35	Account-unbound hashes allow digest collisions	Signatures	● Medium	Resolved
M-36	Disabled markets block accrued funds	DoS	● Medium	Resolved
M-37	Deposit reader overquotes capped positive impact	Unexpected Behavior	● Medium	Resolved
M-38	Relay Fee Atomic Swap Extraction	Oracle	● Medium	Resolved
L-02	minAmountOut Ignored In BridgeOut	Logical Error	● Low	Resolved
L-05	GLV Oracle Count Mismatch Causes Keeper Loss	Gas Griefing	● Low	Resolved
L-07	Sync Lacks Parameter/baseKey Binding	Validation	● Low	Resolved
L-13	Tier Change Could Cause Reverts For Fee Workflow	Logical Error	● Low	Resolved
L-14	Broad OFT Trigger For BRIDGED_GMX_RECEIVED	Configuration	● Low	Resolved
L-21	Protocol-Unfavorable Rounding Review	Rounding	● Low	Resolved

Main Review Findings & Resolutions (continued)

ID	Title	Category	Severity	Status
L-22	Unsigned prices control median in GMOracle	Oracle	● Low	Resolved
L-23	Subaccounts can self-deal fee-routing fields	Validation	● Low	Acknowledged
L-24	Equal max impact caps remove safety margin	Configuration	● Low	Resolved
L-25	TWAP tranche splits strand vault dust	Rounding	● Low	Resolved
L-26	SUI and WAVAX retain impact-pool drips	Configuration	● Low	Resolved
I-01	Outdated Comments In updateAdlState	Documentation	● Info	Resolved
I-02	Redundant Index Price Fetch In Withdrawal	Gas Optimization	● Info	Resolved
I-04	Incorrect CRE Network/Config Values	Configuration	● Info	Resolved
I-06	Fee Rewards Manipulation Via Snapshot Inflation	Gaming	● Info	Resolved
I-07	Implementation mismatch for forceCalculation	Documentation	● Info	Resolved
I-08	pnlAfterPricelImpactUsd Calculated Twice	Superfluous Code	● Info	Resolved
I-09	Market TWAP Orders Always Revert	Logical Error	● Info	Resolved
I-10	Missing Multichain TwapOrder Functionality	Logical Error	● Info	Resolved

Main Review Findings & Resolutions (continued)

ID	Title	Category	Severity	Status
I-11	External-driven max funding cap can lag GMX	Logical Error	● Info	Resolved
I-12	TWAP orders inherit fixed creation price bounds	Logical Error	● Info	Resolved
I-13	TWAP orders inherit fixed creation price bounds	Logical Error	● Info	Resolved
I-14	TWAP bypasses tightening of creation validations	Logical Error	● Info	Resolved

C-01 | Reserve check dangerously blocks decrease flows

Category	Severity	Location	Status
Logical Error	● Critical	PositionUtils.sol, MarketUtils.sol	Resolved

Description

When `sizeDeltaUsd < 0`, `PositionUtils.updateOpenInterest` performs an additional absolute backing check:

```
| if (sizeDeltaUsd < 0) {  
|     uint256 poolUsd = MarketUtils.getPoolUsdWithoutPnl(...);  
|     uint256 reservedUsd = MarketUtils.getReservedUsd(...);  
|     if (reservedUsd > poolUsd) {  
|         revert Errors.InsufficientReserve(reservedUsd, poolUsd);  
|     }  
| }
```

This check is much stricter than the configured reserve-factor gates used elsewhere in the system.

In all markets checked, `reserveFactor` and `openInterestReserveFactor` are greater than 100%. That means the protocol explicitly allows positions to be opened while `reservedUsd` is well above `poolUsd`, as long as `reservedUsd` remains below the configured reserve-factor ceilings.

However, once a user later tries to reduce position size, the transaction must satisfy the stricter `reservedUsd <= poolUsd` rule. This is a direct inconsistency: the protocol accepts one reserve invariant on entry, then enforces a different and stricter invariant on exit.

Example scenario:

- `poolUsd = $100m`
- `reserveFactor = 200%`, so `validateReserve` allows up to `$200m`
- A user opens or increases positions while market-wide `reservedUsd = $150m`
- This state is valid for entry, because `$150m <= $200m`

Recommendation

Consider removing this newly introduced check entirely.

Resolution

GMX: Resolved.

C-02 | factory.call() enables unauthorized transfers

Category	Severity	Location	Status
Signatures	● Critical	SignatureUtils.sol:44-L45	Resolved

Description

SignatureUtils.validateSignature() supports EIP-6492 by executing attacker-controlled calldata during signature validation in SignatureUtils.sol:154 and SignatureUtils.sol:167. This path is reachable from relay authorization in BaseGelatoRelayRouter.sol:391.

The call is executed by the relay router itself, and the relay routers are granted ROUTER_PLUGIN and CONTROLLER privileges in deployGelatoRelayRouter.ts:23, deploySubaccountGelatoRelayRouter.ts:30, and roles.ts:62.

As a result, an attacker can supply factory and factoryCalldata that invoke privileged protocol functions during signature validation, before authentication is completed.

Root Cause

For EIP-6492 signatures, _validateEIP6492Signature() decodes (factory, factoryCalldata, originalSignature) and performs:

```
(bool success, ) = factory.call(factoryCalldata);
```

For undeployed-wallet deployment and again for the deployed-wallet retry path.

There is no restriction that:

- Factory is an approved deployment factory
- FactoryCalldata is a deployment call
- The call is side-effect-free
- The call cannot target privileged protocol contracts

This is unsafe because the caller of factory.call(...) is the relay router, not the user.

Recommendation

Do not allow arbitrary calls during signature validation.

Resolution

GMX: Resolved.

H-06 | depositReferralRewards Call Would Always Revert

Category	Severity	Location	Status
Rounding	● High	feeDistributionService.ts:983-L984	Resolved

Description

In distribute(), the on-chain contract computes a single aggregate WNT amount and stores it:

```
| // FeeDistributor.sol line 679-680
| uint256 scaledWntPrice = _getUint(Keys2.FEE_DISTRIBUTOR_WNT_PRICE) *
Precision.FLOAT_PRECISION;
| uint256 wntForReferralRewards = Precision.toFactor(wntReferralRewardsInUsd,
scaledWntPrice);
| This is effectively floor(totalRebateUsd / wntPrice) – one division on the aggregate.
```

In the subsequent depositReferralRewards step, the off-chain code re-queries the subgraph and computes per-affiliate and per-trader amounts individually:

```
| // feeDistributionService.ts line 935
| const amount = rebateUsd / wntPrice; // per affiliate
| // feeDistributionService.ts line 974
| // unchanged lines omitted
| if (totalWntAmount !== wntAmount) {
|   throw new Error(...)
```

Due to integer division, $\text{sum}(\text{floor}(x_i / d))$ is always less than or equal to $\text{floor}(\text{sum}(x_i) / d)$. Each individual division can lose up to $\text{wntPrice} - 1$ in remainder. With N affiliates and traders, the total difference can be up to $N * (\text{wntPrice} - 1)$.

Example

3 Affiliates, wntPrice = 2818733868580000 (ETH at \$2818.73, 12 extra decimals):

Off-chain sum: 354847484485902594 + 709694968971805188 + 177423742242951297 = 1241966195700659079
On-chain: $\text{floor}(3500 * 10^{30} / 2818733868580000) = 1241966195700659081$

Difference: 2 wei With only 3 affiliates the difference is 2 wei. In production with hundreds of affiliates and traders, the cumulative truncation loss grows proportionally.

Recommendation

Consider applying division on the sum like its done in first step of the distribute or Replace the strict equality check with a tolerance-based comparison that accounts for the maximum possible truncation loss.

Resolution

GMX: Resolved.

H-07 | Wrong Operation Order Cause Reward Loss

Category	Severity	Location	Status
Logical Error	● High	FeeDistributor.sol:618	Resolved

Description

In `FeeDistributor._distributeFees()`, `updateLastDistributionTime()` is called before `setTokensPerInterval()`:

```
| IRewardDistributor(distributor).updateLastDistributionTime(); // sets lastDistributionTime =  
| IRewardDistributor(distributor).setTokensPerInterval(feeAmountGmx / 1 weeks);
```

This ordering causes pending rewards from the previous interval to be silently lost. The call chain is:

- `updateLastDistributionTime()` sets `lastDistributionTime = block.timestamp`
- `setTokensPerInterval()` internally calls `IRewardTracker(rewardTracker).updateRewards()`
- `updateRewards()` calls `_updateRewards()` which calls `IRewardDistributor(distributor).distribute()`
- `distribute()` calls `pendingRewards()`, which computes:

```
|     uint256 timeDiff = block.timestamp.sub(lastDistributionTime);  
|     return tokensPerInterval.mul(timeDiff);
```

Since `lastDistributionTime` was already set to `block.timestamp` in step 1, `timeDiff == 0`, and `pendingRewards()` returns `0`.

- `distribute()` early-returns with `0` – no rewards are transferred.

As a result, all accrued rewards from the previous interval (since the last distribution) are lost and never distributed to stakers.

Recommendation

Remove the `updateLastDistributionTime()` call entirely. It is redundant because `setTokensPerInterval()` already triggers the full distribution flow: `setTokensPerInterval()` → `updateRewards()` → `distribute()`, and `distribute()` internally sets `lastDistributionTime = block.timestamp` after transferring pending rewards.

```
| - IRewardDistributor(distributor).updateLastDistributionTime();  
|   IRewardDistributor(distributor).setTokensPerInterval(feeAmountGmx / 1 weeks);
```

Resolution

GMX: Resolved.

H-10 | Non-Determinism Issues In Chainlink CRE Workflow

Category	Severity	Location	Status
Logical Error	● High	gmx-functions (global)	Acknowledged

Description

Chainlink CRE runs workflows across multiple nodes that must reach consensus. All nodes execute the same code and must produce identical results. If any node produces a different output, consensus fails and the workflow doesn't execute.
Reference: <https://docs.chain.link/cre/concepts/non-determinism-ts>

The Problem

The workflow has multiple sources of non-determinism that cascade into consensus failure:

```
| new Date() → different query time ranges per node
|
|   ↓
| No orderBy on subgraph queries → different/unordered results per node
| // unchanged lines omitted
|
|   ↓
| No fees distributed. No referral rewards deposited.
```

Sources of Non-Determinism

- new Date() instead of trigger timestamp
- fee-automation-workflow/utils/date.ts:4 — getRecentWednesdayStartOfDay() uses new Date() to compute the weekly period for all subgraph queries. The CRE trigger already provides a deterministic payload.scheduledExecutionTime(main.ts:201) but it is never used.

If nodes execute across a Wednesday midnight UTC boundary, they compute different weekly periods and query entirely different data:

```
| Node | new Date() | Computed Wednesday | | --- | --- | --- | | Node A | Wed Jan 8 00:00:00 | Jan 8 | | Node B | Tue Jan 7 23:59:59 | Jan 1 (previous week) |
```

Recommendation

Consider following all best practices mentioned here <https://docs.chain.link/cre/concepts/non-determinism-ts>

- Use trigger timestamp — replace new Date() with payload.scheduledExecutionTime or block timestamp
- Add orderBy + orderDirection to all subgraph queries

Resolution

GMX: Acknowledged.

H-12 | Reserve checks can block liquidations and ADLs

Category	Severity	Location	Status
Logical Error	● High	PositionUtils.sol, DecreasePositionUtils.sol	Resolved

Description

`PositionUtils.updateOpenInterest` re-runs `validateReserve` and `validateOpenInterestReserve` after applying a negative open interest delta. `DecreasePositionUtils` routes all decreases through this helper, and both liquidation orders and ADL orders execute through the same decrease-order path.

As a result, stressed reductions are not allowed merely because they improve reserve usage. They are only allowed if the entire post-transaction market state is brought fully back within the configured reserve-factor bounds in a single transaction.

This is especially relevant because reserve factors can be configured above 100%. In that configuration, a market can remain fully backed in absolute terms while still violating the configured reserve-factor caps. If a liquidation or ADL reduces exposure but the post-state remains above those caps, the transaction still reverts.

```
| function updateOpenInterest(  
|     PositionUtils.UpdatePositionParams memory params,  
|     int256 sizeDeltaUsd,  
|     // unchanged lines omitted  
|     if (sizeDeltaUsd < 0) {  
|     // unchanged lines omitted
```

Illustrative scenario:

- `poolUsd` = \$100m
- `reserveFactor` = 200%, so `validateReserve` allows up to \$200m
- `openInterestReserveFactor` = 200%, so `validateOpenInterestReserve` allows up to \$200m
- Current `reservedUsd` = \$210m (due to upward price movement in a popular market)

Recommendation

First reconsider if having these validations is important, if they are then Consider

- Only enforcing these absolute checks on positive open-interest deltas.
- For negative deltas, enforce a monotonic rule:
- Post-state violation must be $\leq$ pre-state violation
- And the relevant quantities (`openInterest`, `reservedUsd`) must strictly improve

Resolution

GMX: Resolved.

H-13 | StaticOracleProvider returns min/max inverted

Category	Severity	Location	Status
DoS	● High	StaticOracleProvider.sol:46-L47	Resolved

Description

The StaticOracleProvider returns the min price as max price and the other way around. This likely leads to DoS in the _setPrimaryPrice function.

Recommendation

Return the min price as min price and the max price as max price.

Resolution

GMX: Resolved.

M-01 | Incompatible Gas Assumptions On MegaEVM

Category	Severity	Location	Status
Configuration	● Medium	GasUtils.sol:293-L313	Resolved

Description

MegaETH’s gas model is not singular like EVM. It is multidimensional, with separate costs for compute and storage. Because of this difference, MegaETH recommends using their API for gas estimation instead of relying on standard EVM-style gas calculations.

However, the current implementation uses EVM-based assumptions for gas estimation, which may not accurately reflect actual execution costs on MegaEVM. This affects both keepers and on-chain contract logic.

For example, gas constants and estimations are defined in:
<https://github.com/GuardianOrg/gmx-synthetics-team1-1768586730346/blob/256a058fa996d8da5bea9eb617557e323d7379da/contracts/gas/GasUtils.sol#L293-L313> Or configs around max data length, like data list length.

Megaeth also uses 98/100 rule for gas forwarding instead of 63/64 rule of vanilla EVM.

This could lead to: Underpriced transactions that revert due to insufficient gas Unexpected keeper failures or degraded protocol reliability on MegaEVM

Recommendation

Gas value constants configured for EVM should not be assumed to be valid on MegaEVM. Since MegaEVM does not currently expose a local Forge environment for testing, we recommend deploying to the MegaETH testnet and empirically validating all gas-related constants and estimations. Where possible, integrate MegaETH’s recommended gas estimation API for production usage. (Keepers)

Resolution

GMX: Resolved.

M-07 | validateGlvShiftMaxLoss Uses Post-Shift Input

Category	Severity	Location	Status
Logical Error	● Medium	GlvShiftUtils.sol:236	Resolved

Description

The `validateGlvShiftMaxLoss` check in `executeGlvShift` prices the input and output using post-shift pool values and supply. The shift is executed first, then `getPoolValueInfo` and `getMarketTokenSupply` are queried for both markets. This makes the input side (`marketTokensUsd`) reflect the post-shift price, which can be lower after adverse impact, understating the true cost of tokens removed from `fromMarket`.

```
| cache.marketTokensUsd = MarketUtils.marketTokenAmountToUsd(  
|     glvShift.marketTokenAmount(),  
|     cache.fromMarketPoolValueInfo.poolValue.toUint256(), // using post-shift values  
|     cache.fromMarketTokenSupply  
| );  
| validateGlvShiftMaxLoss(params.dataStore, glvShift.glv(), cache.marketTokensUsd,
```

Illustrative example

- Pre-shift `fromMarket`: pool value \$1000, supply 100 → \$10/GM. GLV shifts 50 GM, so the true cost is \$500.
- The shift worsens pool balance, post-shift pool value drops to \$400 while supply is 50 → \$8/GM.
- If you price the input using post-shift values, those 50 GM look like \$400, not \$500.
- If you received \$350 on the other side, loss looks 12.5% (\$50/\$400) instead of 30% (\$150/\$500).

Impact: the max-loss guardrail can be bypassed in edge cases because the computed loss is smaller than the pre-shift loss, allowing shifts that exceed `glvShiftMaxLossFactor` to pass validation.

Recommendation

Snapshot the `fromMarket` pool value and market token supply before executing the shift and use those pre-shift values for `marketTokensUsd`. Keep `receivedMarketTokensUsd` based on post-shift state, or reuse the withdrawal/deposit valuation inputs to ensure a consistent pre/post comparison.

Resolution

GMX: Resolved.

M-10 | Risk Free Trades For Malicious Keeper

Category	Severity	Location	Status
Validation	● Medium	Config.sol:615	Resolved

Description

In case the LIMITED_CONFIG_KEEPER acts maliciously it is able to perform risk free trades in the following way:

- Create multiple orders with a lot of collateral and a lot of leverage
- Look out if any order would be in instant profit if executed => if so let it go through
- Otherwise update the MAX_OPEN_INTEREST key so that the order can't be executed

Recommendation

Carefully set the allowed keys the LIMITED_CONFIG_KEEPER should be able to update to not open up any potential attack vectors.

Resolution

GMX: Resolved.

M-11 | Event Lookup Across Batch Can Cause Failures

Category	Severity	Location	Status
DoS	● Medium	main.ts:186-L187	Resolved

Description

In `main.ts`, `callFeeDistributor` captures `fromBlock` using the latest header before submitting the transaction, then queries logs from that block onward and expects exactly one matching log. During batched `depositReferralRewards` execution, a later batch can query a range that still includes prior batch logs (or other matching logs in the same range), causing `logs.length > 1` and a hard failure even though the current transaction succeeded.

Recommendation

Consider making log selection transaction-specific instead of range-unique

Resolution

GMX: Resolved.

M-17 | V2 Keeper Costs Can Underflow Calculations

Category	Severity	Location	Status
DoS	● Medium	FeeDistributor.sol:659	Resolved

Description

In `_calculateChainlinkAndTreasuryAmounts`, the treasury amount is calculated by subtracting `keeperCostsV2` from the V2 portion of WNT. However, there is no validation that `keeperCostsV2` fits within the available V2 allocation. If V2 keeper costs exceed `chainlinkTreasuryWntAmount - wntForChainlink`, the subtraction will underflow and revert, blocking the entire weekly fee distribution.

```
| function _calculateChainlinkAndTreasuryAmounts(  
|     uint256 totalWntBalance,  
|     uint256 feesV1Usd,  
|     // unchanged lines omitted  
|     uint256 wntForTreasury = chainlinkTreasuryWntAmount - wntForChainlink - keeperCostsV2;  
|     return (wntForChainlink, wntForTreasury);
```

The `keeperCostsV2` value is calculated in `calculateKeeperCosts` based on the difference between each V2 keeper's target balance and their current native balance.

Unlike V1 costs which have a fallback mechanism in `_finalizeWntForTreasury` that can tap into the treasury allocation or pull from the treasury wallet, V2 keeper costs have no such safety mechanism.

Example scenario:

- V1 fees dominate: `feesV1Usd = 900k`, `feesV2Usd = 100k`
- V2 portion is 10% of total WNT, e.g. 30 ETH
- Chainlink factor takes 10%, leaving 27 ETH for treasury + V2 keepers
- V2 keepers have spent heavily and need 50 ETH top-up
- Calculation: `27 - 50 = underflow -> REVERT`

Recommendation

Options include:

- Cap `keeperCostsV2` to the available V2 allocation (partial top-up)
- Allow V2 keeper costs to also draw from the treasury allocation when the V2 portion is insufficient, mirroring the V1 fallback logic.

Resolution

GMX: Resolved.

M-31 | Atomic Oracle Providers Enable Price Picking

Category	Severity	Location	Status
Logical Error	● Medium	Oracle.sol	Resolved

Description

On live deployments (Arbitrum, Avalanche, Botanix), both `ChainlinkPriceFeedProvider` and `ChainlinkDataStreamProvider` were found to be enabled as atomic providers despite code comments warning against this:

```
| // Oracle.sol#L269
| since for atomic actions, any atomic provider can be used, it is recommended
that only one atomic
```

As a result, users can select the provider that gives them a more favorable price at withdrawal, extracting value from the pool. Furthermore, if the difference in pricing between the two providers is large enough, users can cycle deposits and atomic withdrawals, selecting the favorable provider on each withdrawal and resulting in repeatable profit.

Additionally, the `bridgeOut` flow is wrapped with `withRelay`, which sets prices via `setPricesForAtomicAction` from user-supplied `relayParams.oracleParams`. Users can pick whichever provider yields a more favorable swap rate for the relay fee conversion, reducing their effective relay fee input and shifting the economic loss onto GMX swap liquidity / impact pools over repeated use.

Recommendation

Enforce exactly one atomic oracle provider per token at validation time (same strictness as non-atomic path), or disallow mixed atomic providers globally.

Resolution

GMX: Resolved.

M-32 | GLV market token deposits bypass expiry

Category	Severity	Location	Status
Logical Error	● Medium	ExecuteGlvDepositUtils.sol:192	Resolved

Description

GLV deposits where `isMarketTokenDeposit == true` do not enforce per-request oracle timestamp bounds:

```
| minTimestamp >= updatedAtTime
| maxTimestamp <= updatedAtTime + REQUEST_EXPIRATION_TIME
```

- A GLV deposit request stores `updatedAtTime`.
- For non-market-token GLV deposits:
- Execution follows the synthetic Deposit flow.
- This path inherits `ExecuteDepositUtils` timestamp and expiration checks.
- For market-token GLV deposits:
- Execution enters `_processMarketDeposit()`.

A keeper can execute a market-token GLV deposit using oracle timestamps that would be considered invalid under normal request-level freshness rules.

This creates an inconsistency between market-token and non-market-token GLV deposit validation and may allow execution with stale or out-of-bounds oracle data.

Recommendation

Consider adding request-relative timestamp validation at the start of `ExecuteGlvDepositUtils.executeGlvDeposit(...)` (or immediately before market-token early return).

Resolution

GMX: Resolved.

M-33 | Edge data stream decimals are not validated

Category	Severity	Location	Status
Validation	● Medium	EdgeDataStreamProvider.sol: 74-L80	Resolved

Description

EdgeDataStreamProvider.getOraclePrice uses edgeDataStreamTokenDecimalsKey(token) to scale prices, but does not validate this value before applying:

```
| uint256 tokenDecimals =
dataStore.getUint(Keys.edgeDataStreamTokenDecimalsKey(token));
| int256 floatMultiplier = int256(30) - int256(tokenDecimals) + report.expo;
| uint256 adjustedBidPrice = report.bid * 10 ** (uint256(floatMultiplier));
```

If edgeDataStreamTokenDecimalsKey(token) is misconfigured (for example 0), output prices can be mis-scaled by orders of magnitude. In comparison, ChainlinkDataStreamProvider scales by dataStreamMultiplierKey(token) which explicitly reverts when unset:

```
| uint256 multiplier = dataStore.getUint(Keys.dataStreamMultiplierKey(token));
| if (multiplier == 0) {
|     revert Errors.EmptyDataStreamMultiplier(token);
```

This misconfiguration was found in live deployments for these tokens: ZORA, KTA, XPL, ASTER, 0G, AVNT, LINEA e.g.

Recommendation

- In EdgeDataStreamProvider.getOraclePrice, revert if token decimals are unset.
- This issue should have been mitigated operationally because TimelockConfig.signalSetEdgeDataStream sets both edgeDataStreamId and edgeDataStreamTokenDecimals in the same scheduled batch.

This issue was only expected if: a. Timelock admin explicitly passes 0 for edgeDataStreamTokenDecimals during signalSetEdgeDataStream (governance/operator error), or

Resolution

GMX: Resolved.

M-34 | Missing Chainlink ref price feeds for tokens

Category	Severity	Location	Status
Configuration	● Medium	Oracle.sol	Resolved

Description

Oracle.sol implements a defense-in-depth mechanism where, for non-Chainlink-on-chain providers (i.e. ChainlinkDataStreamProvider, EdgeDataStreamProvider, GmOracleProvider), it cross-validates the provider's returned price against a Chainlink on-chain price feed:

```
| if (!provider.isChainlinkOnChainProvider()) {
|     (bool hasRefPrice, uint256 refPrice) =
ChainlinkPriceFeedUtils.getPriceFeedPrice(dataStore, token);
|     if (hasRefPrice) {
|         _validateRefPrice(token, validatedPrice.min, refPrice,
maxRefPriceDeviationFactor);
|         _validateRefPrice(token, validatedPrice.max, refPrice,
maxRefPriceDeviationFactor);
|     }
```

If a price feed is not configured (priceFeedKey(token) == address(0)), getPriceFeedPrice returns (false, 0) and the validation is silently skipped. This means the primary oracle provider's price is accepted without any secondary sanity check.

An on-chain configuration audit found 9 tokens that have Chainlink price feeds deployed and active on Arbitrum, but are not configured in GMX's DataStore: XRP, LTC, SHIB, STX, POL, SUI, SEI, APT, TON.

Avalanche and Botanix were also audited. No additional tokens with available-but-unconfigured Chainlink feeds were found on those chains.

The reference price check bounds how far the primary provider's price can deviate before the transaction reverts. Without it, a manipulated data stream, compromised keeper, or provider scaling bug would be accepted unconditionally.

Recommendation

Configure priceFeedKey, priceFeedMultiplierKey, and priceFeedHeartbeatDurationKey in the DataStore for all tokens where a Chainlink price feed exists on the deployed chain.

Resolution

GMX: Resolved.

M-35 | Account-unbound hashes allow digest collisions

Category	Severity	Location	Status
Signatures	● Medium	BaseGelatoRelayRouter.sol, RelayUtils.sol	Resolved

Description

Multichain actions use global replay protection keyed only by digest, while many multichain struct hashes do not bind account. This lets two different users produce the same digest for the same payload shape.

_validateCall burns digest before signature validation:

```
| bytes32 digest = ECDSA.toTypedDataHash(domainSeparator, structHash);
| _validateDigest(digest); // consumed globally
| RelayUtils.validateSignature(domainSeparator, digest, relayParams.signature, account,
"call");
```

For multiple multichain actions, the struct hash omits account, e.g. getBridgeOutStructHash:

```
| return keccak256(
|     abi.encode(
|         BRIDGE_OUT_TYPEHASH,
|         // unchanged lines omitted
|     )
| );
```

Illustrative example

- Bob submits signed bridgeOut payload P.
- Alice copies P, signs as Alice, and submits first.
- Alice tx succeeds, consumes digest(P).
- Bob tx reverts with InvalidUserDigest.

Affected actions:

Recommendation

Include account in every multichain struct hash that currently omits it, including claims and referral actions. Keep the digest unique per user action.

Resolution

GMX: Resolved.

M-36 | Disabled markets block accrued funds

Category	Severity	Location	Status
DoS	● Medium	MarketUtils.sol	Resolved

Description

`claimFundingFees` and `claimCollateral` now both end with `validateMarketTokenBalance` after transferring funds out of the market token contract. In both flows, `market` is passed as an address, so Solidity resolves to the overload that first calls `getEnabledMarket`. That helper reverts with `DisabledMarket` whenever the market has been disabled.

```
| function claimFundingFees(  
|     DataStore dataStore,  
|     EventEmiter eventEmiter,  
| // unchanged lines omitted  
|     validateEnabledMarket(dataStore, market);  
|     return market;
```

Illustrative example

- A market is disabled as an emergency response to oracle issues or operational instability.
- A user already has `claimableCollateralAmount` or accrued funding fees recorded for that market from a prior decrease or settlement flow.
- The user calls `claimCollateral` or `claimFundingFees`.
- The transaction reverts with `DisabledMarket`, so the user cannot withdraw the already-accrued claim while the market remains disabled.

Impact

- Users can be frozen out of withdrawing already-accrued claimable collateral and funding fees.

Recommendation

Do not require a market to be enabled when validating post-claim token balances for passive withdrawals.

Resolution

GMX: Resolved.

M-37 | Deposit reader overquotes capped positive impact

Category	Severity	Location	Status
Unexpected Behavior	● Medium	ReaderDepositUtils: 166, ExecuteDepositUtils: 409	Resolved

Description

ReaderDepositUtils.getDepositAmountOutForSingleToken does not model capped positive deposit price impact the same way as on-chain deposit execution.

Execution only uses the tokenOut impact pool:

```
| if (_params.priceImpactUsd > 0) {  
|     (int256 positiveImpactAmount, /* cappedDiffUsd */) =  
MarketUtils.applySwapImpactWithCap(...);  
|     mintAmount += MarketUtils.usdToMarketTokenAmount(  
| // unchanged lines omitted  
|         marketTokensSupply  
|     );  
| }
```

If tokenOut cannot cover the full positive impact, execution ignores the capped remainder.

The reader does more. It falls back to the tokenIn impact pool and increases amountIn:

```
| if (params.priceImpactUsd > 0) {  
|     (int256 positiveImpactAmount, uint256 cappedDiffUsd) =  
MarketUtils.getSwapImpactAmountWithCap(...);  
|     if (cappedDiffUsd != 0) {  
|         (int256 tokenInPriceImpactAmount, /* cappedDiffUsd */) =  
|         amountIn += tokenInPriceImpactAmount.toUint256();  
|     }
```

This fallback exists in SwapUtils for swaps, but not in ExecuteDepositUtils for deposits. As a result, Reader.getDepositAmountOut can return a mint amount that no real deposit can receive.

Illustrative example

- A deposit earns \$100 of positive impact.

Recommendation

Remove the tokenIn fallback from ReaderDepositUtils so the reader only quotes value that deposit execution can actually mint

Resolution

GMX: Resolved.

M-38 | Relay Fee Atomic Swap Extraction

Category	Severity	Location	Status
Oracle	● Medium	BaseGelatoRelayRouter.sol, SubaccountGelatoRelayRouter.sol	Resolved

Description

Relay fee handling exposes an atomic swap path before the wrapped action-specific logic executes. `_handleRelayFee` pulls `feeToken` from the user and swaps it into WNT through GMX using `AtomicSwap` pricing under user-supplied atomic oracle params.

```
| if (relayParams.fee.feeSwapPath.length != 0) {  
|     _sendTokens(account, relayParams.fee.feeToken, address(contracts.orderVault),  
|     RelayUtils.swapFeeTokens(contracts, eventEmitter, oracle, relayParams.fee));
```

`swapFeeTokens` is a real protocol swap. It uses `swapPricingType = AtomicSwap`, and sends the WNT output to the relay router itself. After the wrapped action returns, the router pays only the actual relay fee and refunds the remaining WNT back to the user.

```
| uint256 residualFeeAmount = ERC20(contracts.wnt).balanceOf(address(this));  
| relayFee = GasUtils.payGelatoRelayFee(..., residualFeeAmount);  
| residualFeeAmount -= relayFee;  
| if (residualFeeAmount > 0) {  
|     _transferResidualFee(contracts.wnt, account, residualFeeAmount, srcChainId);
```

The crux is:

- The attacker chooses stale or mispriced atomic oracle prices to used for the fee conversion
- The protocol executes a real `feeToken` -> WNT swap using those prices
- The resulting WNT is refunded back to the attacker except for the true relay fee
- There is no general cap on relay fee swap size for this path, so if a favorable oracle mispricing occurs the attacker can scale the swapped notional to maximize extraction

Recommendation

Considering that relay swap is a quality-of-life feature, we would recommend removing the feature entirely. If removal is not feasible, considering implementing swap size caps and setting higher atomic swap fees

Resolution

GMX: Resolved.

L-02 | minAmountOut Ignored In BridgeOut

Category	Severity	Location	Status
Logical Error	● Low	LayerZeroProvider.sol:315	Resolved

Description

LayerZeroProvider.prepareSend overwrites the user-provided minAmountOut with Stargate’s quoted amountReceivedLD, effectively disabling user slippage protection for bridge-out operations:

```
| // prepareSend
|         sendParam = SendParam({
|             dstEid: _dstEid,
| // unchanged lines omitted
|         (, , receipt) = stargate.quoteOFT(sendParam);
|         sendParam.minAmountLD = receipt.amountReceivedLD;
```

For example:

- User submits a signed bridge-out with minAmountOut = X.
- prepareSend replaces minAmountLD with receipt.amountReceivedLD.
- The call proceeds even if receipt.amountReceivedLD < X, which is contrary to the signed intent.

As a result, users (and controller flows) cannot enforce a minimum received amount on bridge-out.

Recommendation

Consider removing this param or document the risk for users.

Resolution

GMX: Resolved.

L-05 | GLV Oracle Count Mismatch Causes Keeper Loss

Category	Severity	Location	Status
Gas Griefing	● Low	GlvDepositUtils.sol:159	Resolved

Description

GLV deposit creation validates execution fees assuming `glvTokenPriceUsed` is false, yielding an `oraclePriceCount` of $2 + \text{marketCount} + \text{swapsCount}$. However, keepers typically provide a GLV oracle price during execution, resulting in `glvTokenPriceUsed` being true and an `oraclePriceCount` of $4 + \text{swapsCount}$. For GLVs with exactly 1 market, the creation estimate (3) is lower than the execution value (4).

```
| // Creation - always assumes false
| uint256 oraclePriceCount = GasUtils.estimateGlvDepositOraclePriceCount(
|     marketCount,
|     swapPathLength,
|     false
| );
|
| // Execution - uses actual value
| cache.oraclePriceCount = GasUtils.estimateGlvDepositOraclePriceCount(
|     cache.marketCount,
|     // unchanged lines omitted
| );
| GasUtils.payExecutionFee(..., cache.oraclePriceCount, ...);
```

On every execution of a single-market GLV deposit, keeper payment is calculated using `oraclePriceCount` of 4 but capped to the execution fee collected for 3. The keeper loses `EXECUTION_GAS_FEE_PER_ORACLE_PRICE` (250,000 gas) worth of compensation on every single-market GLV deposit execution, regardless of success or failure.

Recommendation

In `GlvDepositUtils.createGlvDeposit`, use the maximum of both oracle price count calculations during fee validation:

```
| uint256 oraclePriceCount = GasUtils.estimateGlvDepositOraclePriceCount(
|     marketCount,
|     // unchanged lines omitted
| );
| GasUtils.validateExecutionFee(dataStore, estimatedGasLimit, params.executionFee,
oraclePriceCount);
```

Resolution

GMX: Resolved.

L-07 | Sync Lacks Parameter/baseKey Binding

Category	Severity	Location	Status
Validation	● Low	ConfigSyncer.sol	Resolved

Description

In ConfigSyncer.sol, the sync function trusts the additionalData baseKey and does not verify that it matches the parameter string used to fetch the oracle update. The keeper enforces this mapping off-chain, but the on-chain contract does not. If a bad oracle update is emitted, a direct sync can apply a different baseKey than the parameter label implies.

```
| IRiskOracle.RiskParameterUpdate memory u =
|     IRiskOracle(riskOracle).getLatestUpdateByParameterAndMarket(parameter, market);
| (bytes32 baseKey, bytes memory data) = abi.decode(u.additionalData, (bytes32,
bytes));
| _validateMarketInData(baseKey, market, data);
| _validateKey(baseKey);
| config.setUint(baseKey, data, Cast.bytesToUint256(u.newValue));
```

Example scenario:

- Parameter = maxOpenInterestForLongs
- AdditionalData = (baseKey = POSITION_IMPACT_FACTOR, data = abi.encode(market, true))
- Parameter does not match baseKey!
- Result: position impact factor set to a very large value, enabling impact pool drain (see comments MarketUtils.sol#L898)

Recommendation

Bind parameter to an expected baseKey and data format on-chain, and revert if the decoded baseKey or data schema does not match the parameter. This preserves the off-chain intent even if sync is called directly.

Resolution

GMX: Resolved.

L-13 | Tier Change Could Cause Reverts For Fee Workflow

Category	Severity	Location	Status
Logical Error	● Low	feeDistributionService.ts:983-L984	Resolved

Description

The tier query in getAffiliatesTierQuery (feeDistributionService.ts line 231) has no time filter — it queries current on-chain tier assignments:

```
Affiliates(where: { tierId_in: ["2", "1"] })
```

Both distribute and depositReferralRewards call getDistributionData(), which runs this query independently. Since tier assignments can change on-chain at any time, if an affiliate's tier is updated between the two steps, the recalculation in depositReferralRewards will produce different totals than what distribute stored on-chain.

Validation checks fail:

- TotalEsGmxAmount !== esGmxAmount (feeDistributionService.ts line 958) → revert
- TotalWntAmount !== wntAmount (feeDistributionService.ts line 983) → revert

Recommendation

Consider snapshotting the tier data during the distribute step or modify subgraph to consider time period for tier returns.

Resolution

GMX: Resolved.

L-14 | Broad OFT Trigger For BRIDGED_GMX_RECEIVED

Category	Severity	Location	Status
Configuration	● Low	main.ts:418-L419	Resolved

Description

The workflow listens for OFTReceived on the GMX adapter and routes to bridgedGmxReceived. This can trigger on any qualifying OFT receipt matching the filter, not strictly the fee-distribution bridge hop. A third party may be able to cause extra trigger executions by sending bridge traffic that satisfies the filter.

Impact

If CRE charges per trigger/run, this can become a meaningful cost/griefing vector. If trigger cost is negligible and handler exits cheaply, impact is lower.

Recommendation

Tighten trigger constraints to distribution-specific contex.

Resolution

GMX: Resolved.

L-21 | Protocol-Unfavorable Rounding Review

Category	Severity	Location	Status
Rounding	● Low	DecreasePositionCollateralUtils.sol, PositionPricingUtils.sol	Resolved

Description

• In `payForCost`, when remaining costs are paid using `secondaryOutputToken`, required token amount is computed with floor division:

```
| remainingCostInSecondaryOutputToken = remainingCostInOutputToken * collateralTokenPrice.min /
```

This can round required payment down. In edge cases, the required secondary amount becomes 0 even though a positive USD cost remains, and `remainingCostUsd` is later set to 0, marking fees as fully paid.

DecreasePositionCollateralUtils.sol:611

• Borrowing fee tokenization uses floor division:

```
| borrowingFees.borrowingFeeAmount = borrowingFeeUsd / collateralTokenPrice.min
```

Tokenized borrowing fees can be slightly lower than intended USD-equivalent fees.

PositionPricingUtils.sol:477

• UI fee tokenization uses floor division:

UI fee amounts can be slightly under-collected in token terms.

PositionPricingUtils.sol:526

• Position fee tokenization uses floor division:

Protocol position fees can be slightly under-collected in token terms.

PositionPricingUtils.sol:568

Recommendation

Consider round-up conversion or accept/document this as intentional dust behavior.

Resolution

GMX: Resolved.

L-22 | Unsigned prices control median in GMSOracle

Category	Severity	Location	Status
Oracle	● Low	GMSOracleProvider.sol	Resolved

Description

GMSOracleProvider.getOraclePrice decodes a report from caller-supplied data and derives signers from report.signerInfo. Both the sorting check and the signature validation iterate only signers.length times, but the median price computation operates over the full price arrays. There is no check that these array lengths equal signers.length.

```
| // Sorting check – bounded by signers.length
| for (uint256 i = 0; i < signers.length; i++) {
|     if (i == 0) { continue; }
| // unchanged lines omitted
| uint256 medianMinPrice = Array.getMedian(report.minPrices) * (10 ** report.precision);
| uint256 medianMaxPrice = Array.getMedian(report.maxPrices) * (10 ** report.precision);
```

Array.getMedian picks the middle element of the array it receives. Since the sorting check only covers indices [0, signers.length), elements beyond that are completely unvalidated.

Example Scenario: Assume MIN_ORACLE_SIGNERS = 3 and the three legitimate signers report valid min prices [100, 200, 300].

A malicious keeper constructs a report with:

- signerInfo encoding 3 signers
- signatures: 3 valid signatures (+ any padding)
- minPrices: [100, 200, 300, 999999, 999999, 999999, 999999]

Result:

- Sorting check passes – only validates [100, 200, 300].

Recommendation

Add an array length check at the start of getOraclePrice, after deriving signers:

```
| if (report.minPrices.length != signers.length ||
|     report.maxPrices.length != signers.length ||
| // unchanged lines omitted
|     report.signatures.length
| );
```

Resolution

GMX: Resolved.

L-23 | Subaccounts can self-deal fee-routing fields

Category	Severity	Location	Status
Validation	● Low	SubaccountUtils.sol	Acknowledged

Description

The codebase already treats subaccounts as a distinct trust boundary and adds explicit protections where a delegated subaccount could misuse the main account's funds.

For example, relay fee swaps are subject to a dedicated subaccount cap because a malicious subaccount could create a value-extracting swap against the main account:

```
| if (relayParams.fee.feeSwapPath.length != 0) {
|     if (isSubaccount) {
|         // a malicious subaccount could create a large swap with a negative price impact
|         // unchanged lines omitted
|         uint256 maxRelayFeeSwapUsd =
|         // unchanged lines omitted
```

However, subaccount order validation only constrains receiver and cancellationReceiver:

```
| function validateCreateOrderParams(
|     address account,
|     IBaseOrderUtils.CreateOrderParams calldata params
| // unchanged lines omitted
|     revert
|     // uiFeeReceiver and referralCode are left unconstrained
```

The remaining fee-routing fields (uiFeeReceiver and referralCode) are then used as-is when the order is created. This lets a delegated subaccount redirect economic value from the main account's trades to itself.

Example

- Alice authorizes a trading bot as a subaccount.

Recommendation

Extend subaccount-specific validation to cover all value-bearing fee-routing fields, not just token receivers.

Resolution

GMX: Acknowledged.

L-24 | Equal max impact caps remove safety margin

Category	Severity	Location	Status
Configuration	● Low	markets.ts, DecreasePositionCollateralUtils.sol	Resolved

Description

The Arbitrum SOL [SOL-SOL] market is configured with equal max positive and max negative position impact caps.

```
| SOL [SOL-SOL] (0xf22CFFA7B4174554FF9dBf7B5A8c01FaaDceA722)
| maxPosImpact: +=0.50% -=0.50%
```

The codebase and README both rely on `max positive < max negative` to keep capped round trips unprofitable. On increase, positive position impact is capped by `capPositiveImpactUsdByMaxPositionImpact`:

```
| cache.priceImpactUsd = MarketUtils.capPositiveImpactUsdByMaxPositionImpact(
|     params.contracts.dataStore,
|     params.market.marketToken,
|     cache.priceImpactUsd,
|     params.order.sizeDeltaUsd()
| );
```

On decrease, negative total impact is capped to the configured max negative value, with the excess moved into `priceImpactDiffUsd`:

This means a trader can open a balance-improving position and immediately close it after the market remains skewed, realizing up to the capped positive impact on entry while only paying up to the capped negative impact on exit. When the two caps are equal, the protocol loses the built-in price-impact spread that is meant to make this round trip structurally lossmaking.

Illustrative example

- Entry would earn +0.9% uncapped impact, but is capped to +0.5%
- Exit would pay -1.4% uncapped impact, but is capped to -0.5%

Recommendation

Set `positiveMaxPositionImpactFactor` strictly below `negativeMaxPositionImpactFactor` for SOL [SOL-SOL].

Resolution

GMX: Resolved.

L-25 | TWAP tranche splits strand vault dust

Category	Severity	Location	Status
Rounding	● Low	OrderUtils.sol	Resolved

Description

createTwapOrder splits the total collateral and execution fee across twapCount orders using integer division:

```
| params.numbers.initialCollateralDeltaAmount = initialCollateralDeltaAmountTwap / twapCount;  
| // unchanged lines omitted  
| params.numbers.executionFee = wntAmount / twapCount;
```

When either total is not exactly divisible by twapCount, the remainder is not assigned to any child order. The individual orders are created using only the floored per-tranche amounts, and there is no refund or residual-accounting step for the leftover units.

This is visible in the test_replay_TWAP_EQ_03 replay: the TWAP path and the equivalent individual path end with identical sizeInUsd and sizeInTokens, but the TWAP path reports 2 fewer units of collateral because the split rounds down during tranche creation.

As a result, small dust amounts can remain in OrderVault after TWAP order creation. Since that remainder is not tied to any order, it becomes unattributed balance that can accumulate over time.

Impact

- Users may pay slightly more total collateral or execution fee than the created TWAP orders actually consume
- Leftover dust can build up in OrderVault across repeated TWAP usage

Recommendation

Track and handle the TWAP split remainder explicitly. A simple fix is to assign the remainder to the final tranche.

Resolution

GMX: Resolved.

L-26 | SUI and WAVAX retain impact-pool drips

Category	Severity	Location	Status
Configuration	● Low	Global	Resolved

Description

The Arbitrum SUI [WETH-USDC] market and the Avalanche WAVAX [WAVAX-USDC] market are configured with nonzero `positionImpactPoolDistributionRate` values even though the surrounding peer markets use 0. Both stand out as exceptions in their respective market sets and may have been left behind after the broader position impact pool distribution settings were disabled for other markets.

```
| // Arbitrum Chain
| SUI [WETH-USDC] (0x6Ecf2133E2C9751cAAAdCb6958b9654baE198a797)
| positionImpactPoolRate/day:    1082.999999
| // unchanged lines omitted
| positionImpactPoolRate/day:    143.424000
| minPositionImpactPoolAmount:  141000000000000000000
```

`distributePositionImpactPool` is called on deposits, withdrawals, and order execution, so any excess SUI or WAVAX in the position impact pool is gradually released back into pool value instead of remaining available to fund future positive position impact. This changes those markets' behavior relative to peers:

- Positive position impact capacity on SUI and WAVAX decays over time
- GM holders on those markets receive the excess impact-pool value sooner than on zero-rate markets
- Traders on those markets face a different realized price impact profile than traders on otherwise similar markets

This is not a direct exploit, but it is a live economic asymmetry. If the rates were left enabled unintentionally, SUI and WAVAX users are operating under different rebate and pool-value dynamics than the rest of the market set.

Recommendation

Set `positionImpactPoolDistributionRate` for SUI and WAVAX to 0 if the intention is to align them with the other Arbitrum and Avalanche markets. Otherwise, document the exceptions explicitly so the different position impact behavior is intentional and reviewable.

Resolution

GMX: Resolved.

I-01 | Outdated Comments In updateAdlState

Category	Severity	Location	Status
Documentation	● Info	AdlUtils.sol:68-L69	Resolved

Description

The comments in `updateAdlState` state that the `isAdlEnabled` flag is used to "avoid having to repeatedly validate whether auto-deleveraging is required." However, the actual implementation in `executeAdl` always re-validates the PnL factor by calling `isPnLFactorExceeded` regardless of the flag state. The flag only provides early revert when `isAdlEnabled` is false, not gas savings on successful ADL execution paths.

Recommendation

Update comments to accurately reflect that the flag serves as an early revert mechanism and public coordination signal, not a gas optimization for successful ADL executions.

Resolution

GMX: Resolved.

I-02 | Redundant Index Price Fetch In Withdrawal

Category	Severity	Location	Status
Gas Optimization	● Info	ExecuteWithdrawalUtils.sol:457	Resolved

Description

_getOutputAmounts calls params.oracle.getPrimaryPrice for the index token even though prices.indexTokenPrice is already available from getMarketPrices. This duplicates the oracle read and introduces an unnecessary dependency on a second call.

Recommendation

Use the existing prices.indexTokenPrice when calling getPoolValueInfo to keep price usage consistent and reduce redundant oracle reads.

Resolution

GMX: Resolved.

I-04 | Incorrect CRE Network/Config Values

Category	Severity	Location	Status
Configuration	● Info	configArbitrum.json:2-L3	Resolved

Description

ConfigArbitrum.json is configured with mainnet chain identifiers (e.g., Arbitrum mainnet chain name/id) while isTestnet is set to true, and several addresses appear to be placeholders/same-address values.

Recommendation

Consider reviewing all configs before deployment and correcting wherever required.

Resolution

GMX: Resolved.

I-06 | Fee Rewards Manipulation Via Snapshot Inflation

Category	Severity	Location	Status
Gaming	● Info	FeeDistributor.sol:121	Resolved

Description

`initiateDistribute()` determines each chain's share of GMX rewards by reading `spot totalSupply()` from the `extendedGmxTracker` on each chain. This snapshot is stored and used throughout the rest of the distribution cycle to calculate `requiredGmxAmount`:

```
| uint256 stakedGmx = IERC20(extendedGmxTracker).totalSupply();  
| _setUint(Keys2.feeDistributorStakedGmxKey(chainId), stakedGmx);
```

Since `initiateDistribute()` is called on a predictable schedule (Wednesday 0 UTC), an attacker who already holds a stake on a chain can front-run the keeper transaction by temporarily inflating that chain's `totalSupply` — e.g., via flashloaned GMX — and unstaking immediately after the snapshot is taken. Increasing rewards allocation for their chain, and thereby their own stake.

Recommendation

Consider smoothing out the `totalSupply` consideration by using a time-weighted average over the distribution cycle duration

Resolution

GMX: Resolved.

I-07 | Implementation mismatch for forceCalculation

Category	Severity	Location	Status
Documentation	● Info	GlvUtils.sol:37	Resolved

Description

`GlvUtils.getGlvValue` comments state there is a `forceCalculation` option to force basket-based GLV valuation even when a primary GLV oracle price exists.

However, the actual function signature has no such parameter, and logic always short-circuits to the GLV oracle path when `oracle.primaryPrices(glv)` is present.

This creates a spec/implementation mismatch: callers cannot request deterministic basket-based valuation in cases where comments indicate they should be able to.

Recommendation

Implement `forceCalculation` in `getGlvValue` and thread it through relevant call sites. Or remove/update comments to reflect current behavior (no override available).

Resolution

GMX: Resolved.

I-08 | pnlAfterPriceImpactUsd Calculated Twice

Category	Severity	Location	Status
Superfluous Code	● Info	ReaderPositionUtils.sol:259-L272	Resolved

Description

pnlAfterPriceImpactUsd is calculated twice in the getPositionInfo function.

Recommendation

Consider to remove one of the two calculations.

Resolution

GMX: Resolved.

I-09 | Market TWAP Orders Always Revert

Category	Severity	Location	Status
Logical Error	● Info	OrderUtils.sol	Resolved

Description

`createTwapOrder` always assigns a nonzero `validFromTime` to every slice after the first one. Since `_createOrder` explicitly forbids market orders with a nonzero `validFromTime`, any TWAP request using a market order type and `twapCount >= 2` will always revert.

Recommendation

If TWAP market orders should be supported, introduce a different execution model for market orders. Each slice should be stored in a way that allows delayed execution without relying on the normal market-order `validFromTime` restriction.

Resolution

GMX: Resolved.

I-10 | Missing Multichain TwapOrder Functionality

Category	Severity	Location	Status
Logical Error	● Info	GLOBAL	Resolved

Description

The `createTwapOrder` function takes a `srcChainId` as param but can only be used on the local chain.

Recommendation

Consider to let multichain users create TwapOrders or to remove this param.

Resolution

GMX: Resolved.

I-11 | External-driven max funding cap can lag GMX

Category	Severity	Location	Status
Logical Error	● Info	fundingRate.ts:506-L507	Resolved

Description

The keeper computes a target `maxFundingRate` from reference exchange funding magnitudes, then moves toward that target gradually using a time window, periodic execution, thresholds, and max-change limits (`fundingRate.ts` (line 50), `fundingRate.ts` (line 529), `fundingRate.ts` (line 539)).

That target is written on-chain as `MAX_FUNDING_FACTOR_PER_SECOND`, and the contract then bounds effective funding within configured limits (`MarketUtils.sol` (line 1666), `MarketUtils.sol` (line 1672)).

```
f_ps = min(maxFundingFactorPerSecond, x * fundingFactor)
```

If reference venues are relatively balanced while GMX is still meaningfully skewed, the externally derived cap can sit lower than what GMX-local inventory pressure would ideally want.

However since keeper pushes max target down toward floor/buffer smoothly (not instant; windowed + rate-limited), the impact here is not that severe.

Recommendation

Be aware that calm external funding regimes can suppress GMX skew-correction pressure.

Monitor markets that stay skewed for long periods while `maxFundingRate` is pinned near low floors or conservative caps and intervene if needed.

Resolution

GMX: Resolved.

I-12 | TWAP orders inherit fixed creation price bounds

Category	Severity	Location	Status
Logical Error	● Info	Global	Resolved

Description

The TWAP implementation creates all child orders upfront and copies the same pricing fields into every slice:

- TriggerPrice
- AcceptablePrice
- MinOutputAmount

Only validFromTime, collateral, and execution fee are adjusted per child. As a result, later slices execute against future market conditions while still using price constraints chosen at t0.

Later TWAP slices can become:

- Too strict
- AcceptablePrice or minOutputAmount becomes unfillable
- Slices fail, freeze, or remain unexecuted

This undermines the expected behavior of a TWAP, where each slice should reflect market conditions near its scheduled execution time.

Example

Assume a user creates a 5-slice TWAP at t0 with:

If market conditions drift by slice 4:

Even though the user intended to trade over time, slices 4 and 5 are still constrained by bounds chosen 15-20 minutes earlier.

Recommendation

Resolution

GMX: Resolved.

I-13 | TWAP orders inherit fixed creation price bounds

Category	Severity	Location	Status
Logical Error	● Info	Global	Resolved

Description

The TWAP implementation creates all child orders upfront and copies the same pricing fields into every slice:

- TriggerPrice
- AcceptablePrice
- MinOutputAmount

Only validFromTime, collateral, and execution fee are adjusted per child. As a result, later slices execute against future market conditions while still using price constraints chosen at t0.

Later TWAP slices can become:

- Too strict
- AcceptablePrice or minOutputAmount becomes unfillable
- Slices fail, freeze, or remain unexecuted

This undermines the expected behavior of a TWAP, where each slice should reflect market conditions near its scheduled execution time.

Example

Assume a user creates a 5-slice TWAP at t0 with:

If market conditions drift by slice 4:

Even though the user intended to trade over time, slices 4 and 5 are still constrained by bounds chosen 15-20 minutes earlier.

Recommendation

Beware of this.

Resolution

GMX: Resolved.

I-14 | TWAP bypasses tightening of creation validations

Category	Severity	Location	Status
Logical Error	● Info	Global	Resolved

Description

`createTwapOrder()` creates all future slices immediately at `t0` and stores them as ordinary orders. Each child only waits on its own `validFromTime`; it is not re-created when its execution window arrives.

Because of this, every child is only subject to creation-time validations once, when the TWAP is initially submitted. If governance later tightens configuration that is only checked during order creation, already-created future slices are grandfathered in and remain executable.

Creation-only checks include:

- Swap path length validation: `OrderUtils.sol:233`
- Callback gas limit validation: `OrderUtils.sol:267`
- Max data length validation in handler: `OrderHandler.sol:93`
- Execution fee capping under current gas config:

So the issue is specifically that future TWAP slices can bypass later changes to creation-only policy, even though a fresh normal order created at the same future time would revert.

Assume:

- `MAX_CALLBACK_GAS_LIMIT` = 2,000,000 at `t0`
- User creates a 5-slice TWAP with `callbackGasLimit` = 1,500,000

At `t1`, governance lowers:

Now:

Recommendation

Beware of this.

Resolution

GMX: Resolved.

Round Two Findings & Resolutions

ID	Title	Category	Severity	Status
H-01	StrictVault Not Synced In Buyback Flow	Logical Error	● High	Resolved
M-01	M-32 and M-33 fixes introduce build errors	Error	● Medium	Resolved
M-02	GLV market token deposits bypass max expiry	Logical Error	● Medium	Resolved
M-03	Missing init bricks relay fee swaps	Configuration	● Medium	Resolved
M-04	Relay TWAP flow inherits the M-35 hash flaw	DoS	● Medium	Acknowledged
M-05	Zero saved funding biases longs at min	Logical Error	● Medium	Resolved
L-01	Buyback info script reads stale fee holder	Configuration	● Low	Resolved
I-01	Legacy funding key callers write dead slots	Configuration	● Info	Resolved
I-02	Variable Name Inconsistency	Informational	● Info	Resolved
I-03	Missing Batch TWAP functionality	Informational	● Info	Acknowledged
I-04	Unresolved findings from main review	Informational	● Info	Resolved

H-01 | StrictVault Not Synced In Buyback Flow

Category	Severity	Location	Status
Logical Error	● High	FeeHandler.sol:178	Resolved

Description

- The buyback flow does no sync the GMX or WNT amount in the FeeVault. This can lead to an inflated withdrawableBuybackTokenAmount:
- Eve calls `claimFees` to increase the `buybackAvailableFeeAmount` of USDC by \$500 and the FeeVault is synced via `recordTransferIn` to hold \$500 USDC
 - Eve calls `buyback` to swap \$100 WNT to USDC this will decrease the synced USDC amount of the FeeVault as `transferOut` is called but will not sync the WNT balance of the FeeVault as `recordTransferIn` is not used
 - It will also decrease `buybackAvailableFeeAmount` by the USDC amount that Eve will receive and increase the `withdrawableBuybackTokenAmount` by the received WNT tokens
 - Eve calls `claimFees` again but this time with WNT as the `feeToken` which will call `recordTransferIn` to get the amount of unsynced tokens in the FeeVault that equal the current claimable tokens + the WNT received during the last buyback call (that were not synced)
 - Now the `withdrawableBuybackTokenAmount` is increased not only by the claimed amount but also again by the received WNT tokens from the buyback call which will double count them now

This leads to a DoS of the fee distribution flow as it will try to pull more tokens than there are in the Vault.

Recommendation

Call `recordTransferIn` in the buyback flow to keep the FeeVault synced.

Resolution

GMX: Resolved.

M-01 | M-32 and M-33 fixes introduce build errors

Category	Severity	Location	Status
Error	● Medium	ExecuteGlvDepositUtils.sol, TimelockConfig.sol	Resolved

Description

Two recent fixes introduce compile-time regressions.

First, 9bc91653 adds a new check in `ExecuteGlvDepositUtils`, but references `deposit.updatedAtTime()` before `deposit` is declared. In this branch, the only in-scope request object is `glvDeposit`.

```
| if (glvDeposit.isMarketTokenDeposit()) {
|     if (params.oracle.minTimestamp() < deposit.updatedAtTime()) {
|         revert Errors.OracleTimestampsAreSmallerThanRequired(
|             params.oracle.minTimestamp(),
|             glvDeposit.updatedAtTime()
|         );
|     }
```

Second, 436e2901 changes `InvalidEdgeDataStreamDecimals` to take two arguments in `Errors`, but `TimelockConfig` still reverts with only one:

```
| error InvalidEdgeDataStreamDecimals(address token, uint256 decimals);
| // unchanged lines omitted
| if (edgeDataStreamTokenDecimals == 0) {
|     revert Errors.InvalidEdgeDataStreamDecimals(token);
| }
```

These changes make the patched code uncompileable until both call sites are corrected.

Recommendation

Replace `deposit.updatedAtTime()` with `glvDeposit.updatedAtTime()` in `ExecuteGlvDepositUtils`.

Update the `TimelockConfig` revert to pass both required arguments to `InvalidEdgeDataStreamDecimals`, for example `token` and `edgeDataStreamTokenDecimals`.

Resolution

GMX: Resolved.

M-02 | GLV market token deposits bypass max expiry

Category	Severity	Location	Status
Logical Error	● Medium	ExecuteGlvDepositUtils.sol	Resolved

Description

Commit 9bc91653 was intended to fix M-32, but it only added the lower-bound oracle timestamp check to the GLV `isMarketTokenDeposit` branch. The normal deposit path also enforces `oracle.maxTimestamp <= updatedAtTime + REQUEST_EXPIRATION_TIME`, while the GLV market-token shortcut still returns before applying that expiry check.

```
| if (glvDeposit.isMarketTokenDeposit()) {
|     if (params.oracle.minTimestamp() < glvDeposit.updatedAtTime()) {
|         revert Errors.OracleTimestampsAreSmallerThanRequired(
| // unchanged lines omitted
|         glvVault.transferOut(glvDeposit.market(), glvDeposit.glv(),
glvDeposit.marketTokenAmount());
|         return glvDeposit.marketTokenAmount();
|     }
```

As a result, an old GLV market-token deposit can still execute after expiry as long as the keeper supplies fresh-enough prices for `minTimestamp`. The equivalent normal deposit would revert.

Recommendation

Complete the M-32 fix by validating that `oracle.maxTimestamp <= glvDeposit.updatedAtTime() + REQUEST_EXPIRATION_TIME`

Resolution

GMX: Resolved.

M-03 | Missing init bricks relay fee swaps

Category	Severity	Location	Status
Configuration	● Medium	BaseGelatoRelayRouter.sol, general.ts	Resolved

Description

7b6f71e7de1e30e19b0ae6e62d3b1f384e80b221 adds a new global guard in BaseGelatoRelayRouter that rejects any relay fee swap whose USD value exceeds MAX_RELAY_FEE_SWAP_USD. This is intended to cap oracle mispricing extraction for all relay calls, but the commit only updates the runtime check and tests. It does not update the repo's standard general-config plumbing to initialize the new key in production deployments.

As a result, MAX_RELAY_FEE_SWAP_USD remains at the DataStore default value of 0 unless operators manually seed it out-of-band. Any relay action that uses feeSwapPath will then revert as soon as relayFeeUsd > 0.

```
| if (relayParams.fee.feeSwapPath.length != 0) {
|     uint256 relayFeeUsd = relayParams.fee.feeAmount *
|     // New global cap; defaults to 0 unless explicitly configured
|     uint256 maxRelayFeeSwapUsd = contracts.dataStore.getUint(Keys.MAX_RELAY_FEE_SWAP_USD);
|     if (relayFeeUsd > maxRelayFeeSwapUsd) {
|         revert Errors.MaxRelayFeeSwapExceeded(relayFeeUsd, maxRelayFeeSwapUsd);
|     }
```

The surrounding config scripts still only handle MAX_RELAY_FEE_SWAP_USD_FOR_SUBACCOUNT, so the new global key is not part of the normal rollout path.

Impact

- Breaks all relay calls that rely on swapping fee tokens into WNT until operators manually configure the new key.
- Creates a rollout regression even though tests pass, because the tests explicitly seed MAX_RELAY_FEE_SWAP_USD in their setup.

Recommendation

Add MAX_RELAY_FEE_SWAP_USD to the standard general-config path and initialize it during deployment or migration, instead of relying on manual post-deploy writes.

At minimum:

- Add a maxRelayFeeSwapUsd field to config/general.ts
- Write the value in updateGeneralConfigUtils

Resolution

GMX: Resolved.

M-04 | Relay TWAP flow inherits the M-35 hash flaw

Category	Severity	Location	Status
DoS	● Medium	GelatoRelayRouter.sol, RelayUtils.sol, BaseGelatoRelayRouter.sol	Acknowledged

Description

M-35 identified that non-subaccount relay actions can share the same globally consumed digest because replay protection is keyed only by digest, while some struct hashes do not bind account.

The new non-subaccount `createTwapOrder` flow added in `17c104fa` inherits the same issue. `GelatoRelayRouter.createTwapOrder` builds its digest with `RelayUtils.getCreateTwapOrderStructHash(relayParams, params, twapCount, interval)`, and that overload hardcodes `account = address(0)`:

```
| function getCreateTwapOrderStructHash(  
|     IRelayUtils.RelayParams calldata relayParams,  
|     IBaseOrderUtils.CreateOrderParams memory params,  
|     // unchanged lines omitted  
| ) external pure returns (bytes32) {  
|     return _getCreateTwapOrderStructHash(relayParams, bytes32(0), address(0), params,  
|     twapCount,
```

At the same time, `_validateCall` still burns the digest globally before signature validation:

```
| bytes32 digest = ECDSA.toTypedDataHash(domainSeparator, structHash);  
| _validateDigest(digest);  
| SignatureUtils.validateSignature(  
|     // unchanged lines omitted  
|     // unchanged lines omitted  
| );
```

As a result, two different users signing the same non-subaccount TWAP payload shape produce the same digest. If one user submits first, the other user's transaction reverts with `InvalidUserDigest`, even though the signatures are for different accounts.

Illustrative example

Recommendation

In `createTwapOrder` include `account` in the non-subaccount `getCreateTwapOrderStructHash` path so the digest is unique per user action.

Resolution

GMX: Acknowledged.

M-05 | Zero saved funding biases longs at min

Category	Severity	Location	Status
Logical Error	● Medium	MarketUtils.sol	Resolved

Description

Introduced in commit 8dc3041, the adaptive funding path getNextFundingFactorPerSecond now treats nextSavedFundingFactorPerSecond == 0 as long-side funding when selecting the per-side min and max bounds:

```
| bool isLongFunding = cache.nextSavedFundingFactorPerSecond >= 0;
| configCache.minFundingFactorPerSecond = dataStore.getUintValueFromDataStore(
|     Keys.minFundingFactorPerSecondKey(market.marketToken, isLongFunding)
| // unchanged lines omitted
|     Keys.maxFundingFactorPerSecondKey(market.marketToken, isLongFunding)
| );
```

That would only be a tie-break if the function then kept 0 neutral. It does not. The final min-bound call uses Calc.boundMagnitude, which gives zero a positive sign and upgrades it to +min whenever the selected minimum is non-zero:

```
| cache.nextSavedFundingFactorPerSecondWithMinBound = Calc.boundMagnitude(
|     cache.nextSavedFundingFactorPerSecond,
|     configCache.minFundingFactorPerSecond,
| // unchanged lines omitted
|     cache.nextSavedFundingFactorPerSecond
| );
```

As a result, a neutral saved funding state can be converted into a positive funding regime even when there is no long-side skew. This is reachable in the adaptive branch because equal open interest only early-returns when fundingIncreaseFactorPerSecond == 0. When adaptive funding is enabled, the function continues even if longOpenInterest == shortOpenInterest.

For example:

- fundingIncreaseFactorPerSecond != 0

Recommendation

Treat nextSavedFundingFactorPerSecond == 0 as neutral rather than implicitly long.

Resolution

GMX: Resolved.

L-01 | Buyback info script reads stale fee holder

Category	Severity	Location	Status
Configuration	● Low	printBuybackInfo.ts	Resolved

Description

printBuybackInfo still assumes that buyback inventory is physically held by FeeHandler. After the fee custody refactor, that assumption is no longer true: claimFees, buyback, and withdrawFees all move assets through FeeVault instead.

The script now reads withdrawableBuybackTokenAmount from DataStore but compares it against the token balance of feeHandler.address. On deployments that actually use FeeVault, this can report an artificial shortfall even when the buyback inventory is fully backed.

```
| const datastore = await hre.ethers.getContract("DataStore");
| const feeHandler = await hre.ethers.getContract("FeeHandler");
| const withdrawableAmount = await
| // unchanged lines omitted
| if (contractBalance.lt(withdrawableAmount)) {
|   throw new Error(Insufficient contract balance);
```

Recommendation

Update printBuybackInfo.ts to read balances from FeeVault instead of FeeHandler.

Resolution

GMX: Resolved.

I-01 | Legacy funding key callers write dead slots

Category	Severity	Location	Status
Configuration	• Info	AdaptiveFunding.ts, Config.ts, Keys.sol, keys.ts	Resolved

Description

The funding bound keys were changed from a market-scoped layout to a per-side layout by adding `isLong` to `minFundingFactorPerSecondKey` and `maxFundingFactorPerSecondKey`. This is consistent with the new funding logic, which reads separate long and short bounds. However, some callers still use the legacy one-argument form or still encode config writes with `["address"]` instead of `["address", "bool"]`.

Those stale callers no longer update the active storage slots. They write a dead key while the protocol continues reading the new per-side keys, so the intended funding bounds remain unchanged.

```
| await datastore.setUint(keys.minFundingFactorPerSecondKey(ethUsdMarket.marketToken), 0);
| await datastore.setUint(keys.maxFundingFactorPerSecondKey(ethUsdMarket.marketToken),
decimalToFloat(1));
| // Legacy helper shape: writes the old slot, not the active per-side slots
```

The live funding logic now reads side-specific values:

```
| bool isLongFunding = cache.nextSavedFundingFactorPerSecond >= 0;
| configCache.minFundingFactorPerSecond = datastore.getUintValueFromDataStore(
|     Keys.minFundingFactorPerSecondKey(market.marketToken, isLongFunding)
| // unchanged lines omitted
|     Keys.maxFundingFactorPerSecondKey(market.marketToken, isLongFunding)
| );
```

There is also a stale config test path that still writes the legacy encoding, resulting in broken tests:

Recommendation

- Update all `minFundingFactorPerSecondKey` and `maxFundingFactorPerSecondKey` calls to pass `isLong`
- Replace `legacy encodeData(["address"], [market])` writes with `encodeData(["address", "bool"], [market, isLong])`
- Where the same bound should apply to both sides, write the `true` and `false` keys explicitly

Resolution

GMX: Resolved.

I-02 | Variable Name Inconsistency

Category	Severity	Location	Status
Informational	● Info	RelayUtils.sol:43-L50	Resolved

Description

The CREATE_TWAP_ORDER_TYPE ends with _TYPE while all the others end with _PARAMS.

Recommendation

Consider to use a consistent variable name scheme to follow best practices.

Resolution

GMX: Resolved.

I-03 | Missing Batch TWAP functionality

Category	Severity	Location	Status
Informational	● Info	GLOBAL	Acknowledged

Description

The batch functions do not allow TWAP orders.

Recommendation

Consider to implement TWAP orders into the batch flows, but be aware that for subaccount actions this would require to sum up the twapCount.

Resolution

GMX: Acknowledged.

I-04 | Unresolved findings from main review

Category	Severity	Location	Status
Informational	● Info	Global	Resolved

Description

These findings from the main review remain unresolved:

H-07: Wrong Operation Order Cause Reward Loss H-13: StaticOracleProvider returns min/max inverted
M-17: V2 Keeper Costs Can Underflow Calculations M-31: Atomic Oracle Providers Enable Price Picking
M-35: Account-unbound hashes allow digest collisions L-05: GLV Oracle Count Mismatch Causes Keeper
Loss L-07: Sync Lacks Parameter/baseKey Binding L-21 to L-26

Recommendation

Resolve the issues or mark them as acknowledged.

Resolution

GMX: Resolved.

Round Three Findings & Resolutions

ID	Title	Category	Severity	Status
H-01	StaticOracle freeze affects other live markets	Logical Error	● High	Resolved
M-01	V2 keeper top-ups can DoS distribution	DoS	● Medium	Resolved
M-02	EIP6492_DEPLOYER Address Not Set	Configuration	● Medium	Resolved
M-03	Static oracle unusable as oracle is not enabled	Logical Error	● Medium	Resolved
M-04	Static prices unusable when ref price diverge	Logical Error	● Medium	Resolved
M-05	Static freeze only works with atomic price	Logical Error	● Medium	Resolved
L-02	MIN_CHANGE_DELAY Can DoS Oracle Updates	DoS	● Low	Resolved
I-01	GMOracle Code Remains After Provider Removal	Superfluous Code	● Info	Resolved

H-01 | StaticOracle freeze affects other live markets

Category	Severity	Location	Status
Logical Error	● High	Config.sol, StaticOracleProvider.sol	Resolved

Description

StaticOracleProvider is intended to support freezing conditions for a market that is being closed. However, the implemented freeze is not market-scoped. It changes the oracle provider for a token globally.

```
| function setStaticPriceForToken() {  
| // unchanged lines omitted  
|     _setOracleProviderForToken(address(oracle), token, staticOracleProvider);  
| // unchanged lines omitted
```

There is no per-market static oracle selector. Market pricing reads indexToken, longToken, and shortToken from the same global oracle. Therefore, freezing WETH, USDC, BTC, or any other shared token for one closed market changes the required oracle provider for all other live markets and flows that use that token.

An honest attempt to freeze one closing market can force unrelated live markets to execute with a stale token price. This can misprice pool value, collateral value, etc.

Recommendation

Do not freeze shared long/short tokens. Ensure that the to-be-frozen index token, is not used in another market.

Consider additional validation that rejects freezing a token if any enabled market, GLV, or swap path outside the target closure set still depends on that token.

Alternatively, consider making static freeze state market-scoped, not token-scoped. Store frozen prices under a market-specific key, such as (market, token), and only use those prices for that market's close path.

Resolution

GMX: Resolved.

M-01 | V2 keeper top-ups can DoS distribution

Category	Severity	Location	Status
DoS	● Medium	FeeDistributor.sol	Resolved

Description

FeeDistributor.distribute can still revert during the final WNT allocation step when V2 keeper top-up costs are larger than the V2-side WNT allocation. The previous remediation avoids the immediate subtraction underflow in _calculateChainlinkAndTreasuryAmounts, but it does not cap keeperCostsV2 or give V2 keeper costs the same treasury fallback path used for V1 keeper and referral costs.

```
| function _calculateChainlinkAndTreasuryAmounts(...) internal view returns (uint256,  
uint256) {  
| // unchanged lines omitted  
|     if (keeperCostsV2 > chainlinkTreasuryWntAmount - wntForChainlink) {  
| // unchanged lines omitted  
|         uint256 wntForTreasury = chainlinkTreasuryWntAmount - wntForChainlink - keeperCostsV2;  
|         return (wntForChainlink, wntForTreasury);
```

In the high keeperCostsV2 branch, the function returns the whole non-Chainlink V2 bucket as wntForTreasury while keeperCostsV2 is still paid in full later through wntForKeepers. This can overcommit the same totalWntBalance.

The overcommit is hit in _finalizeWntForTreasury, before V1 fallback logic can help:

```
| function _finalizeWntForTreasury(...) internal returns (uint256) {  
|     uint256 wntBeforeV1KeeperCostsAndReferralRewards = totalWntBalance -  
|         keeperCostsV2 -  
|         wntForChainlink -  
|         wntForTreasury; // can still underflow after the remediation  
| // unchanged lines omitted
```

Illustrative example

- totalWntBalance = 100 WNT.
- V2 receives a 10 WNT allocation based on the feesV1Usd / feesV2Usd split.

Recommendation

Make the V2 keeper shortfall handling explicit and ensure the WNT allocation cannot exceed totalWntBalance.

Resolution

GMX: Resolved.

M-02 | EIP6492_DEPLOYER Address Not Set

Category	Severity	Location	Status
Configuration	● Medium	GLOBAL	Resolved

Description

deployEIP6492Deployer.ts deploys EIP6492Deployer, but skips setting EIP6492_DEPLOYER in DataStore on existing mainnet deployments because the deployer account cannot call Config.setAddress.

The generic config helper can support this update, and EIP6492_DEPLOYER is allowlisted in Config, but no committed script currently adds this config item for the deployed address.

Recommendation

Consider to add an explicit script or config item that sets the EIP6492_DEPLOYER address through the existing config flow after deployment.

Resolution

GMX: Resolved.

M-03 | Static oracle unusable as oracle is not enabled

Category	Severity	Location	Status
Logical Error	● Medium	Config.sol, updateOracleProviders.ts	Resolved

Description

Config.setStaticPriceForToken is intended to activate the static oracle flow for closed-market freezes. It sets the frozen token price, then routes the token to StaticOracleProvider.

In the current deployment configuration, StaticOracleProvider is deployed and passed into Config, but it is not enabled as an oracle provider. updateOracleProviders.ts only enables the live providers:

```
| const providersToAdd = [  
|   chainlinkPriceFeedProvider.address,  
|   chainlinkDataStreamProvider.address,  
|   edgeDataStreamProvider.address,  
| ];
```

Therefore, calling setStaticPriceForToken reverts when it tries to assign staticOracleProvider through _setOracleProviderForToken which rejects any provider that is not globally enabled:

```
| if (!dataStore.getBool(Keys.isOracleProviderEnabledKey(provider))) {  
|   revert Errors.InvalidOracleProvider(provider);  
| }
```

As a result, the intended static-price freeze path is unusable unless operators first enable StaticOracleProvider, otherwise setStaticPriceForToken reverts with InvalidOracleProvider(staticOracleProvider).

Recommendation

An immediate fix would be to enable StaticOracleProvider. However, consider if the setStaticPriceForToken should call isOracleProviderEnabledKey(staticOracleProvider) in the first place as the static oracle is not a normal live provider.

Resolution

GMX: Resolved.

M-04 | Static prices unusable when ref price diverge

Category	Severity	Location	Status
Logical Error	● Medium	Oracle.sol, StaticOracleProvider.sol	Resolved

Description

StaticOracleProvider is intended to support frozen pricing for markets that are being closed. However, static prices are still validated against the live Chainlink reference price when a reference feed exists for the token.

StaticOracleProvider explicitly reports that it is not a Chainlink on-chain provider:

```
| function isChainlinkOnChainProvider() external pure returns (bool) {  
|     return false;  
| }
```

As a result, Oracle._validatePrices applies the reference-price deviation check to static prices:

```
| if (!provider.isChainlinkOnChainProvider()) {  
|     (bool hasRefPrice, uint256 refPrice) =  
ChainlinkPriceFeedUtils.getPriceFeedPrice(dataStore, token);  
|     if (hasRefPrice) {  
|         _validateRefPrice(token, validatedPrice.min, refPrice, maxRefPriceDeviationFactor);  
|         _validateRefPrice(token, validatedPrice.max, refPrice, maxRefPriceDeviationFactor);  
|     }
```

This means a frozen token price remains executable only while it stays within MAX_ORACLE_REF_PRICE_DEVIATION_FACTOR of the current Chainlink feed. If the live Chainlink price moves far enough away from the frozen price, every non-atomic execution requiring that static token price reverts with MaxRefPriceDeviationExceeded. This can leave positions and liquidity stuck or force governance/operators to change oracle configuration again under emergency conditions.

This will also lead to reverts if the reason behind shutting down the market is a deprecated chainlink feed as in that case calling lastRoundData on the chainlink feed will lead to a revert.

Recommendation

Do not apply the live Chainlink reference-price deviation check to explicitly frozen static prices.

Resolution

GMX: Resolved.

M-05 | Static freeze only works with atomic price

Category	Severity	Location	Status
Logical Error	● Medium	Config.sol	Resolved

Description

Config.setStaticPriceForToken is intended to activate the static oracle flow for markets that need to be closed under frozen pricing. However, the function can only initialize the static price from an atomic oracle provider:

```
| function setStaticPriceForToken(  
|     address token,  
|     OracleUtils.SetPricesParams memory pricesParams  
| ) external onlyConfigKeeper nonReentrant  
withOraclePricesForAtomicAction(pricesParams) {  
| // unchanged lines omitted
```

This means the static freeze path is only usable for tokens that have an enabled atomic provider capable of producing the seed price. Tokens whose normal source is a non-atomic provider, such as ChainlinkDataStreamProvider, cannot be frozen through setStaticPriceForToken unless an additional atomic provider is configured for that token or the data stream provider is marked atomic.

This is a practical issue for markets being delisted because static pricing is most valuable when the live oracle source is expected to become unavailable. For example, if a token only has Chainlink Data Streams configured and that stream is deprecated, operators may be unable to initialize static pricing: before deprecation, setStaticPriceForToken reverts with NonAtomicOracleProvider; after deprecation, there may be no fresh data stream report available to seed the static price.

Recommendation

Do not require the freeze seed price to come only from an atomic provider.

Resolution

GMX: Resolved.

L-02 | MIN_CHANGE_DELAY Can DoS Oracle Updates

Category	Severity	Location	Status
DoS	● Low	Config.sol:163	Resolved

Description

In case a new token / market is created but a mistake happened or an emergency situation occurs and the config keeper wants to freeze the token price to wind down the market again, it might be DoSed, as the ORACLE_PROVIDER_MIN_CHANGE_DELAY needs to pass.

Recommendation

Consider to skip the ORACLE_PROVIDER_MIN_CHANGE_DELAY in case of the StaticOracleProvider.

Resolution

GMX: Resolved.

I-01 | GMSOracle Code Remains After Provider Removal

Category	Severity	Location	Status
Superfluous Code	● Info	GLOBAL	Resolved

Description

GMSOracleProvider was removed, but GM oracle-specific production code remains in the codebase:

- GMSOracleUtils still exists and is only used in a test file: OracleModuleTest.
- Errors.sol still contains GM oracle-specific errors such as GmEmptySigner

Recommendation

Consider to remove unused code.

Resolution

GMX: Resolved.

Round Four Findings & Resolutions

ID	Title	Category	Severity	Status
M-01	ADL Uses Spread-Reduced Prices for Solvency	Validation	● Medium	Resolved
M-02	Capped Negative Impact Creates Collateral Claim	Validation	● Medium	Acknowledged
M-03	ADL nets same-side PnL across collateral pools	Logical Error	● Medium	Resolved
M-04	Positive Impact Cap Can Revert Settlement	Logical Error	● Medium	Acknowledged
M-05	Pending Deposits Use Mutable UI Fee Factor	Validation	● Medium	Resolved
L-01	Capped Chainlink Answers Accepted as Live	N/A	● Low	Acknowledged
L-02	Swap Impact Uses Gross Input Before Fees	Math	● Low	Acknowledged
L-03	Unset cancellationReceiver Misroutes Refunds	Logical Error	● Low	Acknowledged
L-04	Empty Pool Side Freezes Liquidations	Validation	● Low	Acknowledged
L-05	Traders can self-refer to capture tier-0 rebates	Rewards	● Low	Acknowledged
L-06	Dust Fee Payment Redirects Decrease Fees	Rewards	● Low	Acknowledged
L-07	Raw Negative Impact Reverts Before Cap	Logical Error	● Low	Acknowledged
L-08	Withdrawal Swaps Bypass Paused Entry Points	Validation	● Low	Acknowledged

Round Four Findings & Resolutions (continued)

ID	Title	Category	Severity	Status
I-01	Cost Conversion Drops Secondary-Token Dust	Rounding	● Info	Acknowledged
I-02	Increase Orders Use Weaker Timestamp Checks	Validation	● Info	Acknowledged
I-03	Same-Token Markets Can Enable Invalid ADL	Oracle	● Info	Acknowledged
I-04	Sub-Unit Borrowing Fees Reset to Zero	Rounding	● Info	Acknowledged
I-05	Fallback Emits Zero srcChainId	Events	● Info	Acknowledged

M-01 | ADL Uses Spread-Reduced Prices for Solvency

Category	Severity	Location	Status
Validation	● Medium	N/A	Resolved

Description

ChainlinkDataStreamProvider.getOraclePrice() narrows the verified data-stream bid/ask by dataStreamSpreadReductionFactor before Oracle stores min/max prices. ADL has no raw-bounds path: AdlUtils.updateAdlState() loads those primary prices and MarketUtils.isPnlFactorExceeded() calls getPnlToPoolFactor(..., maximize=true).

Impact: When data-stream spreads widen, the raw verified risk bounds can require ADL while on-chain spread-reduced prices keep ADL disabled. During that window profitable traders can close normally and DecreasePositionCollateralUtils deducts their positive PnL from market pool amounts instead of the protocol forcibly deleveraging them, allowing LP liquidity to be drained and potentially leaving the GM market insolvent before ADL activates.

Recommendation

Preserve raw verified data-stream bid/ask separately from spread-reduced execution prices and use raw conservative bounds for ADL, max-PnL-factor and solvency checks. Do not let dataStreamSpreadReductionFactor narrow prices consumed by MAX_PNL_FACTOR_FOR_ADL paths.

Resolution

GMX: Resolved.

M-02 | Capped Negative Impact Creates Collateral Claim

Category	Severity	Location	Status
Validation	● Medium	N/A	Acknowledged

Description

The partial-decrease pre-check models only initialCollateralDeltaAmount as a collateral withdrawal. When negative price impact is capped, the uncapped excess is put in priceImpactDiffUsd. processCollateral later pays that diff from output and then remainingCollateralAmount, but unlike ordinary costs it credits the paid token amount to the trader as claimable collateral.

Impact: The residual position can remain open while posted collateral is lower than the explicit-withdrawal checks would allow, and the removed amount can become externally claimable via claimCollateral after the delay/factor logic kicks in. This increases leverage/free-option risk and can worsen liquidation shortfalls after an adverse move, though the residual must still satisfy the static maintenance factor at execution time.

Recommendation

Treat priceImpactDiffUsd paid from remaining collateral as an effective collateral withdrawal in the pre-check, or prevent the diff from being credited to the trader when doing so would reduce residual collateral below the same thresholds as an explicit withdrawal.

Resolution

GMX: Acknowledged.

M-03 | ADL nets same-side PnL across collateral pools

Category	Severity	Location	Status
Logical Error	● Medium	N/A	Resolved

Description

ADL uses `MarketUtils.getPnlToPoolFactor()`, which divides same-side net PnL by only the long-token pool for longs or the short-token pool for shorts. The numerator from `getPnl()` sums open interest and open-interest-in-tokens across both long-token and short-token collateral positions for the same side. This is not settlement-equivalent: profitable longs are paid from the long-token pool and profitable shorts from the short-token pool (`DecreasePositionUtils` sets `pnlToken = market.longToken/shortToken` by side), while losing positions repay costs into their own collateral-token pool.

Impact: ADL can remain disabled or stop too early for a side whose payout token pool is above the configured PnL limit. Profitable traders can close and reduce that token pool while offsetting losses credit another pool, creating token-specific depletion that can impair PnL settlement and withdrawals and violate the intended ADL risk control. Severity is Medium because the strongest demonstrated impact is incorrect risk-control activation and per-pool imbalance/depletion rather than a complete proven protocol-wide drain.

Recommendation

Compute ADL and trader-PnL-cap exposure using settlement-equivalent per-payout-token accounting. Do not subtract losses that settle into the opposite collateral pool from the positive liability owed by the long-token pool for longs or the short-token pool for shorts; alternatively, model and enforce thresholds on the actual token-level settlement flows before allowing ADL to remain disabled.

Resolution

GMX: Resolved.

M-04 | Positive Impact Cap Can Revert Settlement

Category	Severity	Location	Status
Logical Error	● Medium	N/A	Acknowledged

Description

CapPositiveImpactUsdByPositionImpactPool computes available lendable capacity from the combined USD value of the long-token and short-token pools. Actual settlement in DecreasePositionCollateralUtils pays positive close impact only by decrementing the pnlToken pool. When the pnlToken side is depleted while the opposite side still has liquidity, the cap accepts a positive impact value that applyDeltaToPoolAmount cannot subtract from the pnlToken pool (it reverts via the negative-poolAmount guard).

Impact: Risk-reducing decreases that incur positive close impact in markets where the pnlToken pool is depleted but the opposite side is funded become unexecutable, even though the cap reports sufficient combined liquidity. This produces a transient DoS for affected close orders during imbalanced liquidity states.

Recommendation

Cap positive impact by the amount actually payable from the pnlToken pool (or otherwise source settlement from the same combined liquidity considered by the cap) before calling applyDeltaToPoolAmount.

Resolution

GMX: Acknowledged.

M-05 | Pending Deposits Use Mutable UI Fee Factor

Category	Severity	Location	Status
Validation	● Medium	N/A	Resolved

Description

Deposit requests persist only uiFeeReceiver, not the UI fee factor in effect when the asynchronous request is created, and there is no per-request max UI fee. The receiver can publicly change its factor while deposits are pending. Execution later passes the stored receiver into swap/deposit fee calculation, where SwapPricingUtils.getSwapFees re-reads the receiver's current factor from DataStore. Thus a frontend/receiver can advertise a low fee at request creation and raise the factor before execution, with the pending request charged using the later factor rather than the factor observed by the user at creation.

Impact: Loss is capped by MAX_UI_FEE_FACTOR, but within that cap a malicious chosen UI receiver can extract part of users' slippage buffer from already-pending deposits, or cause tight-slippage deposits to revert and be cancelled, costing execution fees and delaying liquidity access.

Recommendation

Snapshot uiFeeFactor in the deposit request and use that stored value during all deposit swap/mint fee calculations, or add a user-specified maxUiFeeFactor/maxUiFeeAmount bound checked at execution.

Resolution

GMX: Resolved.

L-01 | Capped Chainlink Answers Accepted as Live

Category	Severity	Location	Status
N/A	● Low	N/A	Acknowledged

Description

The Chainlink on-chain price-feed path only requires `latestRoundData().answer` to be positive and within the configured heartbeat. It does not reject answers equal to the underlying aggregator's `minAnswer` or `maxAnswer` circuit-breaker bounds. When a Chainlink aggregator clamps to those bounds because the underlying asset has moved outside the configured range, `ChainlinkPriceFeedProvider` converts the capped answer and returns it as a valid min/max oracle price.

Impact: For tokens whose primary provider is configured to `ChainlinkPriceFeedProvider`, an extreme price move that hits the aggregator's min/max clamp results in protocol pricing (atomic withdrawals, ADL/PnL solvency checks, fee swaps) using a stale bounded value instead of the true market price. This can misprice pool withdrawals and PnL and incorrectly enable or disable ADL during the precise market conditions where the circuit breaker is active. Impact is gated by the on-chain Chainlink provider being selected as primary for a token and by an actual aggregator-bound breach.

Recommendation

Reject capped Chainlink answers. Store configured min/max bounds per feed (or read aggregator bounds via an extended interface) and revert when `latestRoundData().answer` is at or beyond those bounds. Consider also disabling affected markets or requiring an uncapped fallback oracle when a feed cap is reached.

Resolution

GMX: Acknowledged.

L-02 | Swap Impact Uses Gross Input Before Fees

Category	Severity	Location	Status
Math	● Low	N/A	Acknowledged

Description

SwapUtils computes balance improvement and positive price impact from the full gross amountIn before fees. getSwapFees then removes protocol and UI fees from that same amount. In the positive-impact branch the rebate withdrawn from the swap impact pool is still based on the gross priceImpactUsd, while pool accounting credits only amountAfterFees + feeAmountForPool. feeReceiverAmount and uiFeeAmount are claimable liabilities and do not become LP-owned pool liquidity.

Impact: Affected positive-impact swaps receive an extra rebate from swap impact pools for self-reclaimed UI-fee liquidity that never enters pool accounting. The leak is bounded by configured max UI fee factor, swap impact factor, available positive-impact opportunities, and impact-pool balances. The PoC shows a 100-token gross rebalance receiving a 20-token-price-unit rebate even though only 80 tokens are credited to pool accounting after fees; the real pool-credit-based rebate would be 16.

Recommendation

Calculate swap price impact and balanceWasImproved using the actual amount that will be credited to pool accounting, or otherwise exclude protocol/UI fee liabilities from the positive-impact rebate basis. In particular, self-claimable UI fees should not increase the liquidity delta used to pay rebates from swap impact pools.

Resolution

GMX: Acknowledged.

L-03 | Unset cancellationReceiver Misroutes Refunds

Category	Severity	Location	Status
Logical Error	● Low	N/A	Acknowledged

Description

OrderUtils.cancelOrder uses two different fallback recipients when cancellationReceiver is unset: initialCollateralDelta refunds fall back to order.account, but execution-fee refunds fall back to order.receiver. When a user sets receiver to a third party for order output and leaves cancellationReceiver unset, the WNT execution-fee refund goes to the third-party receiver instead of the funding account.

Impact: An account that funds an order with a third-party receiver and does not set cancellationReceiver loses the unused execution-fee refund on cancellation; the refund is sent (or unwrapped and sent as native) to receiver. Bounded to the unused execution fee per order.

Recommendation

Use the same fallback for execution-fee refunds as for collateral refunds: when cancellationReceiver is zero, default to order.account, or require cancellationReceiver to be explicitly set whenever receiver differs from account.

Resolution

GMX: Acknowledged.

L-04 | Empty Pool Side Freezes Liquidations

Category	Severity	Location	Status
Validation	● Low	N/A	Acknowledged

Description

A profitable decrease deducts positive PnL directly from the corresponding pool amount and can reduce that side's pool to zero while leaving residual open interest/reservedUsd. There is no post-deduction reserve/solvency check in the decrease path. Liquidations and ADL executions therefore revert before reducing OI, and deposits revert before adding liquidity, making the state unrecoverable through normal flows.

Impact: The same root cause as the prior ADL-suppression issue escalates into a full market-side freeze. After the reachable zero-pool/nonzero-OI state exists, normal orders, liquidations, ADL executions, and deposits are blocked by the borrowing-accrual pre-hook. Remaining positions can become or already be liquidatable, but risk-reduction cannot execute, so bad debt can grow with adverse price movement until privileged off-path intervention repairs accounting or recapitalizes the pool.

Recommendation

Prevent normal execution from creating zero-pool/nonzero-reserved states by validating reserves/solvency after PnL and impact-pool deductions in decrease processing. Additionally, borrowing accrual should not hard-revert when poolUsd is zero; cap or skip accrual for the depleted side while allowing liquidations, ADL, and deposits to proceed so the market can recover through normal flows.

Resolution

GMX: Acknowledged.

L-05 | Traders can self-refer to capture tier-0 rebates

Category	Severity	Location	Status
Rewards	● Low	N/A	Acknowledged

Description

Production ReferralStorage allows any EOA to register a referral code via registerCode and assign that same code to itself via setTraderReferralCodeByUser without governance approval. ReferralUtils.getReferralInfo and PositionPricingUtils.getPositionFeesAfterReferral only require the referral code to be nonzero — they do not require affiliate != trader, nor that the affiliate has been assigned to a curated tier.

Impact: Any trader can reduce effective position fees by the full configured tier-0 rebate — an upfront trader discount plus a claimable affiliate reward credited to themselves. This is a higher-impact variant of the previously accepted Low 'Unregistered referral codes receive tier-0 rebates' because the affiliate reward is no longer stranded at address(0); it is captured by the trader, reducing protocol/LP fee revenue on all self-referred trading volume.

Recommendation

Disallow self-referrals in ReferralStorage and/or make ReferralUtils.getReferralInfo return zero rebates when codeOwners(code) is address(0) or equals the trader. Consider making tier 0 rebates zero unless an affiliate has been explicitly assigned to a curated, nonzero tier.

Resolution

GMX: Acknowledged.

L-06 | Dust Fee Payment Redirects Decrease Fees

Category	Severity	Location	Status
Rewards	● Low	N/A	Acknowledged

Description

During a decrease, non-funding fees are paid via payForCost from collateral-token output/remaining collateral and then from secondary PnL output if needed. processCollateral only performs normal fee distribution when amountPaidInSecondaryOutputToken is zero. If even one wei is paid in the secondary token, processCollateral credits all paid amounts to poolAmount and replaces PositionFees with getEmptyFees.

Impact: Intended fee recipients (protocol, UI, affiliate, liquidation/borrowing receivers) can be systematically bypassed on decrease orders. A trader with large or flash/transient GM exposure can engineer dust-secondary fee payments to redirect fee revenue into LP pool value and recapture a proportional share, converting external fee revenue into an LP rebate at scale.

Recommendation

Do not zero the entire PositionFees object when any secondary-token fee payment occurs. Split fee accounting by paid token, distribute the collateral-token-paid portion to fee recipients normally, and either create equivalent claimable balances for the secondary-token-paid portion or revert if non-funding fees cannot be paid entirely in the collateral token.

Resolution

GMX: Acknowledged.

L-07 | Raw Negative Impact Reverts Before Cap

Category	Severity	Location	Status
Logical Error	● Low	N/A	Acknowledged

Description

Decrease execution computes raw `priceImpactUsd` and only applies the positive max-impact cap before passing the value to `BaseOrderUtils.getPriceForDecrease`. For negative impact, `BaseOrderUtils` validates the uncapped value and reverts if its magnitude exceeds `sizeDeltaUsd` or if it makes the execution price negative. With the repository's default GMX parameters (0.00001% negative impact factor and quadratic exponent), a \$100k long liquidation in a market with about \$10m opposite-side skew produces about -\$201k raw impact, exceeding the order size.

Impact: In realistically skewed markets, user decreases and liquidations can become unexecutable even though governance configured a max-negative-impact cap intended to bound the charge. Underwater positions can remain open while prices move further against LPs, allowing bad debt to accumulate and preventing the liquidation mechanism from restoring solvency.

Note: The above assumes 0.00001% negative impact factor and quadratic exponent) but most prod markets (checked for Arbitrum and Avalanche) use lower factors and/or exponents below 2, which makes the raw impact grow much slower. No live markets were found to be at risk.

Recommendation

Apply the negative max position impact cap before acceptable-price/execution-price validation, using the same capped `totalImpactUsd` semantics as collateral processing, or move the revert-prone execution-price calculation after negative impact has been capped.

Resolution

GMX: Acknowledged.

L-08 | Withdrawal Swaps Bypass Paused Entry Points

Category	Severity	Location	Status
Validation	● Low	N/A	Acknowledged

Description

Withdrawal requests may include longTokenSwapPath and shortTokenSwapPath. createWithdrawal only checks the withdrawal feature flag, while validateSwapPath checks only path length and enabled swap markets. It does not validate any swap-specific or market-swap order pause.

Impact: Governance lacks a finer-grained lever to pause swap activity routed through withdrawals while still allowing plain LP exits. Containment of an incident currently requires either disabling withdrawal execution outright or disabling the specific swap markets, both of which are broader operational levers than a dedicated swap pause. No concrete fund loss or asymmetric attack primitive was demonstrated; this is a circuit-breaker hardening observation.

Recommendation

Add a dedicated feature flag for withdrawal-embedded swaps, or have non-empty withdrawal swap paths consult the same swap/market-swap pause controls used by direct swap entry points, enforced at both creation and execution.

Resolution

GMX: Acknowledged.

I-01 | Cost Conversion Drops Secondary-Token Dust

Category	Severity	Location	Status
Rounding	● Info	N/A	Acknowledged

Description

PayForCost rounds the USD cost up when converting to collateral/output-token units. Once those sources are exhausted, it converts the remaining collateral-token-denominated cost into secondary-output-token units via floor division ($\text{remainingCostInOutputToken} * \text{collateralPrice} / \text{secondaryPrice}$). Not a duplicate of the positionLastSrcChainId pollution finding – entirely different module (position/DecreasePositionCollateralUtils.payForCost vs. order/OrderUtils._updatePositionLastSrcChainId) with no shared root cause.

Impact: For costs paid from secondary output (funding fees, negative PnL, position fees, negative impact, and impact-diff costs), the unpaid remainder is bounded by less than one smallest unit of the secondary output token – dust-sized in realistic GMX markets and economically uncollectable at scale.

Recommendation

Use round-up division when converting the remaining cost into secondary-output-token units, or compute remainingCostUsd from the original unpaid USD minus the USD value of secondary tokens actually paid so fractional remainders are not discarded.

Resolution

GMX: Acknowledged.

I-02 | Increase Orders Use Weaker Timestamp Checks

Category	Severity	Location	Status
Validation	● Info	N/A	Acknowledged

Description

LimitIncrease and StopIncrease orders can be created with sizeDeltaUsd > 0 and initialCollateralDeltaAmount == 0. During execution, increase orders validate that minOracleTimestamp is at least the order’s updatedAtTime, and for non-market orders, at least validFromTime.

This differs from LimitDecrease and StopLossDecrease, which also require the oracle timestamp to be at least the position’s increasedAtTime. The decrease-side code explicitly handles the case where an order was created before the relevant position existed or before it was last increased.

This is not a creation-time pricing issue: limit and stop orders are priced at execution. Under the expected keeper behavior, execution should use current oracle data, so stale-oracle execution is not expected in practice. The observation is only that the contract does not enforce the same timestamp invariant for conditional increases that it enforces for conditional decreases.

Recommendation

Either document the keeper invariant that conditional order execution uses current oracle data, or align increase-order validation with decrease-order validation.

Resolution

GMX: Acknowledged.

I-03 | Same-Token Markets Can Enable Invalid ADL

Category	Severity	Location	Status
Oracle	● Info	N/A	Acknowledged

Description

For same-token long markets (`indexToken == longToken`), `isPnlFactorExceeded()` calls `getPnlToPoolFactor(..., maximize=true)`, which values long PnL through `getPnl()` at `indexTokenPrice.max` but divides by `getPoolUsdWithoutPnl(..., !maximize)` which values the same-asset pool at `longTokenPrice.min`.

Impact: ADL can be enabled (and other max-PnL risk gates can reject LP actions) for same-token long markets in a narrow factor band where no consistent same-asset oracle price puts the market above the configured threshold. In practice the artifact is bounded by the oracle spread on a single asset (typically a few bps) amplified by OIT/pool, and ADL still executes at fair oracle prices, so the realized harm is limited to marginally premature deleveraging close costs and transient LP-action rejections rather than direct fund loss.

Recommendation

For same-token markets (`indexToken == longToken`), evaluate ADL and `validateMaxPnl` factors with a consistent endpoint for the same asset, e.g. compute the factor at the max/max endpoint (or take the max over min/min and max/max) instead of mixing index max with pool min.

Resolution

GMX: Acknowledged.

I-04 | Sub-Unit Borrowing Fees Reset to Zero

Category	Severity	Location	Status
Rounding	● Info	N/A	Acknowledged

Description

PositionPricingUtils.getBorrowingFees converts borrowingFeeUsd (30-decimal USD) to collateral token units with floor division by collateralTokenPrice.min. If accrued borrowingFeeUsd is below the price of one smallest collateral-token unit, borrowingFeeAmount rounds to zero. Subsequent increase or non-full decrease updates write the latest cumulative borrowing factor to the position, discarding the accrued sub-unit remainder for that interval.

Impact: Per-update discarded amount is strictly bounded to less than one smallest collateral-token unit (sub-1e-18 USD for an 18-decimal \$1 token, up to ~\$0.001 for WBTC-class collateral). Attempting to exploit this requires repeated update transactions, each incurring position fees and gas; no net-positive exploitation has been demonstrated. The rounding behavior is consistent with token-amount accounting throughout GMX.

Recommendation

Either round borrowing fee token amounts up, or accrue and persist sub-unit remainders in USD so they are added to the next fee calculation before the cumulative borrowing factor is advanced on the position.

Resolution

GMX: Acknowledged.

I-05 | Fallback Emits Zero srcChainId

Category	Severity	Location	Status
Events	● Info	N/A	Acknowledged

Description

For a multichain decrease order whose final swap succeeds, the output sent to the multichain vault is recorded with `order.srcChainId()`. If the same final swap reverts, `_handleSwapError` transfers the fallback output to the multichain vault but records it with a hard-coded `srcChainId` of 0. Two otherwise-identical cross-chain decrease proceeds therefore receive different `srcChainId` attribution depending solely on whether the final swap entered the fallback path.

Impact: On-chain multichain balance accounting is unaffected (balances are keyed only by account + token, and bridge-out is user-initiated), but the emitted `MultichainTransferIn` event metadata attributes fallback proceeds to the local chain rather than the order's source chain. This is an event/indexer inconsistency that may complicate off-chain reconciliation.

Recommendation

In `_handleSwapError`, pass `order.srcChainId()` to `MultichainUtils.recordTransferIn` so the fallback path matches the success and secondary-output paths.

Resolution

GMX: Acknowledged.

Round Five Findings & Resolutions

ID	Title	Category	Severity	Status
M-01	ADL Nets Losing Positions Against Pool Liability	Math	● Medium	Acknowledged
M-02	uiFeeFactor Not Snapshotted Everywhere	Logical Error	● Medium	Resolved
M-03	Reference-price Deviation Checks Are Disabled	Oracle	● Medium	Resolved
I-01	Scripts still reference removed GmOracleProvider	Superfluous Code	● Info	Resolved

M-01 | ADL Nets Losing Positions Against Pool Liability

Category	Severity	Location	Status
Math	● Medium	MarketUtils.sol:523-L582	Acknowledged

Description

ADL is intended to determine whether the side-specific payout pool can cover profitable traders. For long positions, positive PnL is paid from the market’s longToken pool; for short positions, positive PnL is paid from the shortToken pool. Losing positions, however, repay losses through their collateral token.

The current `getPositivePnl` function change improves the previous behavior by preventing losses from one collateral-token bucket from offsetting profits in another bucket. However, it still computes net PnL inside each collateral bucket before flooring the result at zero. This means losses can still hide actual payout-pool liability when profitable and losing positions use the same collateral token.

Example in an ETH/USDC market::

- Two ETH long positions use USDC collateral.
- Position A is profitable by \$200, so closing it decreases the ETH pool by \$200.
- Position B is losing by \$150, so closing it increases the USDC pool by \$150.

The current ADL accounting nets these positions and reports only \$50 of positive PnL: `$200 profit - $150 loss = $50 reported liability`

But this is not settlement-equivalent. The ETH pool still needs to pay \$200; the \$150 loss replenishes the USDC pool, not the ETH pool.

Recommendation

A fix would likely require new market variables. If this is not feasible, be aware about that and monitor probably on the keeper side when and which positions to close and adjust the ADL threshold if needed.

Resolution

GMX: Acknowledged.

M-02 | uiFeeFactor Not Snapshotted Everywhere

Category	Severity	Location	Status
Logical Error	● Medium	GLOBAL	Resolved

Description

Normal GM deposits now snapshot uiFeeFactor when the request is created, so the user pays the fee rate that was active when they signed/submitted the deposit, to avoid unexpected slippage and ui fee attacks.

This protection is still missing in all other flows.

Recommendation

Consider to snapshot the uiFeeFactor everywhere.

Resolution

GMX: Resolved.

M-03 | Reference-price Deviation Checks Are Disabled

Category	Severity	Location	Status
Oracle	● Medium	GLOBAL	Resolved

Description

Oracle.sol now requires !provider.isChainlinkOnChainProvider() && provider.shouldCheckRefPrice(). ChainlinkDataStreamProvider and EdgeDataStreamProvider return false on shouldCheckRefPrice. This makes MAX_ORACLE_REF_PRICE_DEVIATION_FACTOR effectively dead for production providers.

Recommendation

Return true for live off-chain providers, and exempt only ChainlinkPriceFeedProvider, static and mock providers.

Resolution

GMX: Resolved.

I-01 | Scripts still reference removed GmOracleProvider

Category	Severity	Location	Status
Superfluous Code	● Info	GLOBAL	Resolved

Description

GmOracleProvider was removed from contracts but some scripts still reference to it.

Recommendation

Consider to remove unused code completely.

Resolution

GMX: Resolved.

Disclaimer

This report is not, nor should be considered, an “endorsement” or “disapproval” of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any “product” or “asset” created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian’s position is that each company and individual are responsible for their own due diligence and continuous security. Guardian’s goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract’s safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

About Guardian

Founded in 2022 by DeFi experts, Guardian is a leading audit firm in the DeFi smart contract space. With every audit report, Guardian upholds best-in-class security while achieving our mission to relentlessly secure DeFi.

To learn more, visit <https://guardianaudits.com>

To view our audit portfolio, visit <https://github.com/guardianaudits>

To book an audit, message <https://t.me/guardianaudits>